Gap-Aware Retrieval (GAR)

A Gap-Aware Reranking Algorithm for Iterative Retrieval-Augmented Generation

Abstract

Retrieval systems commonly rank documents according to their relevance to the original query. This can be insufficient in iterative information-seeking tasks, where the system already has some evidence and needs to retrieve only the information that is still missing. This project proposes **Gap-Aware Retrieval (GAR)**, a reranking algorithm that reorders a first-stage candidate set using three signals: relevance to the question, coverage of the missing information, and novelty with respect to previously collected evidence.

GAR is evaluated inside the existing MIGRES framework, which identifies missing information during iterative retrieval. MIGRES is the experimental framework; a first-stage retriever performs retrieval from the corpus, and GAR is the proposed reranking component applied on top of that retriever's candidates. Which first-stage retriever is used is itself selected empirically (Section 2) rather than fixed in advance; BM25 is one of the candidates compared for that role. The surrounding MIGRES components are kept fixed as far as the original implementation permits. GAR is compared against no reranking and against eleven reranking baselines (AIR, Dartboard/RIG, and RankGPT are adapted to the common-candidate-set setting) — cross-encoder reranking (MonoBERT), MonoT5, gap-aware MonoT5 control (MonoT5-G), pairwise reranking (DuoBERT), ColBERTv2 late-interaction reranking, classical learning-to-rank (LambdaMART), neural learning-to-rank (RankT5), Maximal Marginal Relevance (MMR), coverage-driven reranking (AIR), relevant-information-gain reranking (Dartboard/RIG), and LLM-based listwise reranking (RankGPT) — on the datasets used in the MIGRES evaluation: WikiHop, HotpotQA, MuSiQue, Natural Questions, TriviaQA, and StrategyQA.

The study evaluates retrieval quality, downstream answer quality, ablations, efficiency, and statistical significance. GAR has no trainable parameters in the proposed implementation. Its scoring hyperparameters are selected on a held-out validation set using Bayesian optimization and then frozen before test evaluation. The formulation is motivated by prior work on missing-information-guided retrieval, iterative evidence coverage, relevant information gain, sentence embeddings, and efficient nearest-neighbor search [Wang et al., 2025, Yadav et al., 2020, Pickett et al., 2024, Reimers and Gurevych, 2019, Johnson et al., 2017].

1 Introduction

Retrieval-Augmented Generation (RAG) systems use external documents to provide a language model with information that may not be available in its parameters. A retrieval algorithm therefore has a simple but important job: given a question and a document collection, return a small set of useful documents.

A common retrieval objective is query–document relevance:

$$q + \mathcal{D} \longrightarrow d_1, d_2, \ldots, d_k.$$

This works well when the question can be answered from one retrieval step. It becomes less direct when the system retrieves information iteratively. After one step, some parts of the information need may already be satisfied. The next retrieval step should focus on the remaining gap rather than repeatedly retrieving documents about the same topic.

MIGRES provides a useful setting for this problem. MIGRES is a Missing Information Guided Retrieve-Extraction-Solving framework. It identifies missing information after evidence is collected

and uses that missing information to guide subsequent retrieval [Wang et al., 2025]. In this project, we do not modify the MIGRES reasoning, extraction, or solving framework. Instead, we keep its first-stage retrieval (FSR) unchanged and add GAR as a reranking component applied to the FSR candidate set before the top-k documents are passed on to MIGRES.

The central research question is:

$$\boxed{\text{Does explicitly modeling the current information gap improve retrieval?}}$$

GAR addresses this by scoring a candidate document using:

$$\boxed{S(d \mid q, E_t, G_t) = \alpha R(d, q) + \beta C(d, G_t) + \gamma N(d, E_t)}$$

where R measures relevance, C measures gap coverage, and N measures evidence novelty.

The contribution of this project is the proposed GAR reranking formulation, applied on top of a first-stage retriever's candidates, and its controlled empirical evaluation. Which first-stage retriever is used is selected empirically (Section 2) rather than assumed; GAR's own comparison against reranking baselines (Section 3, Section 8) is run on top of whichever retriever that selection identifies. The individual ideas used to construct GAR are grounded in previous work; the project does not claim that semantic relevance, information coverage, or novelty are individually new.

1.1 What is MIGRES and How Does It Work?

MIGRES stands for *Missing Information Guided Retrieve-Extraction-Solving*. It is an iterative framework designed for questions that require multiple pieces of information. Instead of retrieving documents only once, MIGRES repeatedly checks what information is already available, identifies what is still missing, retrieves information for that gap, and uses the retrieved information to continue solving the question [Wang et al., 2025].

The basic idea is:

$$\boxed{\begin{array}{c}\text{Question} \rightarrow \text{Find Missing Information} \rightarrow \text{Retrieve} \rightarrow \text{Extract} \\ \rightarrow \text{Update Information} \rightarrow \text{Repeat if Needed}\end{array}}$$

In simple terms, MIGRES asks three questions during the process:

1. **What do we already know?** It keeps the evidence collected so far.
2. **What information is still missing?** It identifies the missing information needed to answer the question.
3. **What should we retrieve next?** It uses the missing information to guide the next retrieval step.

For example, suppose a question requires two facts, A and B. After the first retrieval, MIGRES may obtain A but determine that B is still missing. It then focuses the next retrieval step on B:

$$\text{Question} \rightarrow \text{retrieve } A \rightarrow \text{missing } B \rightarrow \text{retrieve } B \rightarrow \text{answer.}$$

MIGRES therefore maintains an iterative state consisting of the original question q, previously collected evidence E_t, and the current missing-information requirements G_t. These are provided to the retrieval component at each iteration:

$$\boxed{(q, E_t, G_t, \mathcal{D})}$$

where $\mathcal{D}$ is the searchable document corpus.

In our work, **MIGRES is the surrounding framework, not our contribution**. We use its iterative missing-information and evidence-generation process, while keeping the first-stage retrieval unchanged and adding GAR as the reranking component applied to the FSR candidate set. MIGRES determines *what information is missing*; GAR determines *which documents are most useful for obtaining that information.*

Thus, the division of responsibility is:

$$\boxed{\underbrace{\text{MIGRES}}_{\text{finds what is missing}} \rightarrow \underbrace{\text{GAR}}_{\text{reranks useful documents}} \rightarrow \underbrace{\text{MIGRES}}_{\text{extracts and updates evidence}}}$$

This allows us to test whether a gap-aware reranking algorithm improves the ranking stage while keeping the rest of the MIGRES framework, including first-stage retrieval, unchanged.

1.2 What We Use from MIGRES

We use MIGRES as the surrounding experimental framework. MIGRES produces the iterative state:

$$(q, E_t, G_t, \mathcal{D}),$$

where:

- q is the original question;
- E_t is evidence collected before retrieval iteration t;
- G_t is the current missing-information state;
- $\mathcal{D}$ is the searchable document corpus.

This state is used in two stages. First, a first-stage retriever performs retrieval from $\mathcal{D}$ using q and each requirement in G_t, producing a candidate set $\mathcal{C}_t$. Second, GAR reranks $\mathcal{C}_t$ using q, E_t, and G_t:

$$T_t = \text{Top}_k\{d \in \mathcal{C}_t : \ S(d \mid q, E_t, G_t)\}\,.$$

MIGRES then continues its normal filtering, extraction, gap-analysis, and solving process.

Throughout this project we write FSR (First-Stage Retriever) for whichever concrete algorithm performs this first stage. Which algorithm FSR refers to is not fixed in advance: Section 2 compares BM25 against several alternative first-stage retrievers, and the best-performing one on validation data is selected and then held fixed for the remainder of the project, so that the GAR-vs-baselines comparison in Section 3 and Section 8 isolates the effect of the reranking method rather than the first-stage retriever.

We therefore compare:

$$\boxed{\text{MIGRES+FSR (no rerank)}} \qquad \text{vs.} \qquad \boxed{\text{MIGRES+FSR+GAR}}.$$

We additionally compare GAR with ten other established reranking methods, each applied on the same FSR candidate set $\mathcal{C}_t$ (Section 3).

1.3 What We Do Not Change

The following MIGRES components should remain fixed wherever the original implementation permits:

- question processing;
- information-gap generation;
- query-generation logic;
- knowledge filtering;
- information extraction;
- answer-generation/solving model;
- stopping conditions;
- prompts and model settings;
- dataset and corpus.

The main experimental variable is therefore the reranking algorithm/method.

2 Information Retrieval Algorithms

2.1 What is an Information Retrieval Algorithm?

An *Information Retrieval (IR) algorithm* is a method used to search a collection of documents and rank the documents according to how useful they are for a given information need. In our setting, the information need is represented by a question or retrieval query q, and the IR algorithm produces an ordered list of documents:

$$\boxed{(q, \mathcal{D}) \rightarrow d_1, d_2, \ldots, d_k}$$

where $\mathcal{D}$ is the document collection and $d_1, \ldots, d_k$ are the top-k retrieved documents.

Different IR algorithms use different signals to decide which documents should be ranked highly. For example, BM25 primarily uses lexical term-matching, DPR and Contriever use dense vector representations, ColBERTv2 uses token-level late interaction, and SPLADE uses sparse neural representations [Robertson and Zaragoza, 2009, Karpukhin et al., 2020, Izacard et al., 2022, Santhanam et al., 2022, Formal et al., 2021].

In a conventional retrieval system, the main objective is:

$$\boxed{\text{Find documents that are relevant to the query.}}$$

However, in an iterative system such as MIGRES, relevance to the original question may not be sufficient. Some information may already have been retrieved, while other information is still missing. This motivates GAR, which additionally considers the current information gap and the evidence already collected.

Thus, the difference between the first-stage retrieval algorithms and GAR is not that GAR is simply another embedding model. GAR is a reranking method whose scoring function explicitly incorporates the current information state, applied to the candidates the first-stage retriever already retrieved:

$$\boxed{\text{Question} + \text{Information Gap} + \text{Existing Evidence} \rightarrow \text{Document Ranking.}}$$

2.2 Why Compare Retrieval Algorithms?

These five methods represent the established landscape of first-stage retrieval, and comparing them is itself a predefined experiment in this project: which one performs best on validation data is selected empirically, not assumed in advance. GAR itself is not one of these five: it is a reranking algorithm applied on top of whichever retriever is selected here, and its own comparison against established reranking algorithms is introduced in Section 3. The primary first-stage retrieval systems considered here are:

BM25, DPR, Contriever, ColBERTv2, SPLADE

These five methods are used through their published or official implementations where possible. Whichever of them achieves the best first-stage retrieval quality on D_{val} (Section 8.9) is selected as FSR and held fixed for the rest of the project — feeding both GAR and the reranking baselines in Section 3 with an identical candidate set. The selection is made under the same candidate-construction procedure that the main experiment later uses: each candidate retriever builds the candidate set of Eq. (9), with one call for q and one for each gap requirement g_i, at a fixed per-query depth, on a recorded reference MIGRES trajectory (Section 8.2). A question-only run is reported alongside it for reference but is not used for selection. Until that selection is made, BM25 is used only as the running illustrative example in the rest of this section and in the worked examples below, because it is the simplest and most standard first-stage method to describe; it is not thereby assumed to be the winner.

2.3 BM25

BM25 is a sparse probabilistic retrieval method based mainly on query-term frequency, inverse document frequency, and document-length normalization [Robertson and Zaragoza, 2009].

A common form is:

$$S_{\text{BM25}}(q,d) = \sum_{t \in q} IDF(t) \frac{f(t,d)(k_1+1)}{f(t,d) + k_1 \left(1 - b + b\frac{|d|}{avgdl}\right)}.$$

Here $f(t,d)$ is the frequency of term t in document d, $|d|$ is document length, $avgdl$ is average document length, and k_1, b are BM25 parameters. A common inverse-document-frequency choice is $IDF(t) = \ln(1 + \frac{N - n_t + 0.5}{n_t + 0.5})$, where N is the number of documents and n_t the number containing t; implementations differ in this and in tokenization, so the exact implementation and parameters must be recorded.

BM25 is especially important to include in this comparison because it is the retrieval baseline used in the original MIGRES paper, which makes it the natural reference point — but this project treats the choice of first-stage retriever as an open empirical question to be settled by the comparison above, rather than adopting the original MIGRES choice unchanged.

2.4 DPR

Dense Passage Retrieval (DPR) uses separate dense encoders for questions and passages and retrieves documents according to their vector similarity [Karpukhin et al., 2020].

The similarity is the dot product of the two encoders' outputs:

$$S_{\text{DPR}}(q,d) = E_Q(q)^T E_P(d),$$

where E_Q and E_P are two independently parameterized BERT encoders and the [CLS] representation is used as the dense vector.

DPR provides a supervised dense-retrieval baseline.

2.5 Contriever

Contriever is an unsupervised dense retrieval method based on contrastive learning [Izacard et al., 2022]. Queries and documents are encoded independently by the same encoder f_θ, whose output is the average of the last-layer hidden states, and the relevance score is their dot product:

$$S_{\text{Contriever}}(q, d) = \langle f_\theta(q), f_\theta(d) \rangle.$$

Contriever provides a dense-retrieval baseline with a different training approach from DPR.

2.6 ColBERTv2

ColBERTv2 uses late interaction. Instead of representing a complete document with only one vector, it keeps token-level representations and computes efficient token-level interactions [Santhanam et al., 2022].

The late-interaction (MaxSim) score is:

$$S_{\text{ColBERT}}(q, d) = \sum_{i \in q} \max_{j \in d} sim(\mathbf{q}_i, \mathbf{d}_j),$$

where $\mathbf{q}_i, \mathbf{d}_j$ are normalized token embeddings and sim is their dot product (equivalently, cosine similarity). ColBERTv2 keeps this scoring rule and changes how the token embeddings are compressed and trained.

This gives a useful comparison with both single-vector dense retrieval and sparse retrieval.

2.7 SPLADE

SPLADE is a neural sparse retrieval method that produces sparse vocabulary representations for queries and documents [Formal et al., 2021].

For an input $t = (t_1, \ldots, t_N)$ with BERT embeddings $(h_1, \ldots, h_N)$, let $w_{ij} = \text{transform}(h_i)^T E_j + b_j$ be the importance of vocabulary token j for input token i, computed from the masked-language-model head. SPLADE aggregates it with a log-saturation:

$$w_j = \sum_{i \in t} \log\left(1 + \text{ReLU}(w_{ij})\right), \qquad \mathbf{s} = (w_j)_{j \in V},$$

and the retrieval score is the dot product of the query and document representations:

$$S_{\text{SPLADE}}(q, d) = \mathbf{s}_q^T \mathbf{s}_d.$$

SPLADE provides a neural sparse baseline.

2.8 Baseline Comparison

System	Retrieval type	Role
MIGRES+BM25	Sparse lexical	Candidate first-stage retriever
MIGRES+DPR	Dense bi-encoder	Candidate first-stage retriever
MIGRES+Contriever	Unsupervised dense	Candidate first-stage retriever
MIGRES+ColBERTv2	Late interaction	Candidate first-stage retriever
MIGRES+SPLADE	Neural sparse	Candidate first-stage retriever

All five are evaluated on D_{val} under the same retrieval-quality metrics (Section 8.9), and established implementations and published checkpoints are used where possible; their exact checkpoint, index, and retrieval parameters must be recorded. The winner of this comparison becomes FSR, held fixed for the rest of the project. All baselines should use established implementations and published checkpoints where possible. GAR does not appear in this table: it is not a first-stage retriever, but a reranking algorithm applied on top of FSR's output. GAR's own comparison, against eleven reranking baselines (three of them adapted; Section 3.14), is given below.

3 Information Reranking Algorithms

3.1 What is Reranking?

A *reranking algorithm* is the second stage of a two-stage retrieval pipeline. A first-stage retriever (denoted FSR; see Section 2) narrows a large corpus $\mathcal{D}$ down to a small candidate set $\mathcal{C}_t$ using a cheap, high-recall method. A reranker then re-scores and reorders only that candidate set using a signal that would be too expensive or too fine-grained to run over the full corpus $\mathcal{D}$ directly:

$$\boxed{(q, \mathcal{C}_t) \to d_1, d_2, \ldots, d_k, \qquad \mathcal{C}_t \subseteq \mathcal{D}, \quad |\mathcal{C}_t| \ll |\mathcal{D}|}$$

Reranking never introduces documents that the first-stage retriever missed; it can only reorder what was already retrieved. This is the key structural difference from a full retrieval algorithm, which must search $\mathcal{D}$ itself.

Example. Suppose the first-stage retriever returns 100 passages for the question "What year was the treaty signed?" because they share lexical terms with the query ("treaty", "signed"). Many of these 100 passages discuss the treaty's causes, its signatories, or its consequences without ever stating the year. A reranker re-examines only these 100 candidates and reorders them so that passages actually containing a year near the word "treaty" are pushed to the top, even if their raw first-stage score was not the highest.

3.2 Why Compare Reranking Algorithms?

GAR reranks the candidate set $\mathcal{C}_t$ returned by FSR, so the correct comparison is against established reranking algorithms operating on the same candidate set, rather than against full first-stage retrievers. The primary systems are:

MonoBERT, MonoT5, DuoBERT, ColBERTv2 (reranker), LambdaMART RankT5, MMR, AIR, Dartboard / RIG, RankGPT, GAR

Each is applied to the identical FSR candidate set $\mathcal{C}_t$, so only the reranking method differs across systems.

3.3 Cross-Encoder Reranking (MonoBERT)

MonoBERT is a pointwise reranker. The query is fed to BERT as the first segment and the candidate passage as the second; the [CLS] vector is passed to a single-layer network that outputs the probability that the candidate is relevant, and every candidate is scored independently [**??**].

$$S_{\text{MonoBERT}}(q, d) = s_d = P(\text{relevant} \mid q, d),$$

trained with the cross-entropy loss $-\sum_{j \in J_{\text{pos}}} \log s_j - \sum_{j \in J_{\text{neg}}} \log(1 - s_j)$. Because the question and document are encoded jointly, MonoBERT captures fine-grained term interactions that bi-encoder retrieval methods cannot, at the cost of requiring a full forward pass per candidate — practical only because $|\mathcal{C}_t|$ is small.

3.4 MonoT5

MonoT5 reframes reranking as a sequence-to-sequence task: the input is `Query:` q `Document:` d `Relevant:` and the model is fine-tuned to generate the token `true` or `false` [**?**].

At inference, a softmax is applied only to the logits $z_{\texttt{true}}$ and $z_{\texttt{false}}$ of the first decoding step, and candidates are ranked by the probability of `true`:

$$S_{\text{MonoT5}}(q, d) = \frac{\exp(z_{\texttt{true}})}{\exp(z_{\texttt{true}}) + \exp(z_{\texttt{false}})}.$$

The original work reports that ranking by the raw `true` logit, or using a softmax over the full vocabulary, performs very poorly, so this two-token normalization is part of the method. MonoT5 provides a pointwise reranking baseline from a different model family than MonoBERT.

3.5 DuoBERT (Pairwise Reranking)

DuoBERT reframes reranking as pairwise comparison: the query and two candidates d_i, d_j are fed to BERT as three segments, and the model estimates the probability that d_i is more relevant than d_j [**?**]:

$$p_{i,j} = P(d_i \succ d_j \mid q).$$

At inference the pairwise probabilities are aggregated into one score per candidate; we use the SUM rule, which is the main configuration of the original work:

$$S_{\text{DuoBERT}}(q, d_i) = \sum_{j \in J_i} p_{i,j}, \qquad J_i = \{j : j \neq i\}.$$

Pairwise comparison can capture relative distinctions between similar candidates that a pointwise score misses, at the cost of $|\mathcal{C}_t|(|\mathcal{C}_t| - 1)$ model calls for a candidate set of size $|\mathcal{C}_t|$. To keep the common candidate-set protocol, DuoBERT is applied directly to the full FSR candidate set $\mathcal{C}_t$, identical to that of every other reranker, with no MonoBERT pre-ranking stage; only the reranking method differs.

3.6 ColBERTv2 as a Reranker

ColBERTv2 keeps token-level representations and computes late-interaction similarity between the question and a candidate document [Santhanam et al., 2022]. In Section 2 it was described as a

first-stage retriever; here the identical scoring function is instead applied only to the FSR candidate set $\mathcal{C}_t$:

$$S_{\text{ColBERT}}(q, d) = \sum_{i \in q} \max_{j \in d} sim(\mathbf{q}_i, \mathbf{d}_j), \qquad d \in \mathcal{C}_t.$$

This provides a direct comparison between a multi-vector late-interaction reranker and GAR's single-score fusion of relevance, coverage, and novelty.

3.7 LambdaMART

LambdaMART is a classical gradient-boosted-tree learning-to-rank algorithm, trained on hand-engineered features (term overlap, document length, prior first-stage retrieval score, and similar signals) rather than learned text representations [Burges, 2010].

$$S_{\text{LambdaMART}}(q, d) = \sum_{\text{trees}} w_{\text{tree}}(\mathbf{x}(q, d)), \qquad d \in \mathcal{C}_t,$$

where $\mathbf{x}(q, d)$ is a hand-engineered feature vector rather than a learned embedding. LambdaMART is included as a non-neural learning-to-rank baseline, since it remains a strong, cheap production reranker in practice and does not depend on the Sentence Transformer model GAR and the other neural baselines share.

3.8 Maximal Marginal Relevance (MMR)

MMR reranks candidates by balancing relevance to the query against redundancy with documents already selected into the output list [Carbonell and Goldstein, 1998]. It is the closest existing baseline to GAR's own relevance-versus-novelty design.

At each step MMR selects the unselected candidate with the largest score,

$$d^\star = \arg \max_{d \in \mathcal{C}_t \setminus S} S_{\text{MMR}}(d),$$

where the score is:

$$S_{\text{MMR}}(d) = \lambda_{\text{MMR}}\, sim(d, q) - (1 - \lambda_{\text{MMR}}) \max_{d_j \in S} sim(d, d_j), \qquad d \in \mathcal{C}_t \setminus S,$$

where S is the set of documents already selected, sim is a similarity function (the original definition allows different similarity functions for the query term and the redundancy term), and $\lambda_{\text{MMR}} \in [0, 1]$ is MMR's own relevance-vs-redundancy trade-off parameter, unrelated to GAR's own scoring weights α, β, γ (Section 5.8), which play no role in this baseline. For the first selected document, when $S = \varnothing$, the redundancy term $\max_{d_j \in S} sim(d, d_j)$ is defined as 0, so the first document is selected using query relevance alone. MMR has no explicit gap-coverage term: it balances only relevance and redundancy, whereas GAR separately scores relevance, gap coverage, and evidence novelty.

3.9 AIR (Coverage-Driven Iterative Evidence Retrieval), Adapted

AIR reformulates its query after each retrieval step by focusing on keywords not yet covered by the evidence already retrieved, so that later steps target complementary evidence rather than repeating earlier ones [Yadav et al., 2020]. AIR directly motivated GAR's coverage term C (Section 4).

In the original method, a candidate P is scored against the current query Q by an IDF-weighted soft alignment over word embeddings:

$$s(Q,P)=\sum_{i=1}^{|Q|}\mathrm{idf}(q_i)\cdot\mathrm{align}(q_i,P),\qquad \mathrm{align}(q_i,P)=\max_{k=1}^{|P|}\cos(q_i,p_k).$$

Query reformulation is driven by the remainder terms, i.e. the query terms not yet covered by the retrieved justifications E_t:

$$Q_r=\tau(Q)\setminus\bigcup_{e\in E_t}\tau(e),$$

where a query term counts as covered if its cosine similarity with some term of e is at least a threshold M (soft matching). The next query is Q_r if $|Q_r|>T$; otherwise Q_r is expanded with the terms of the most recently retrieved justification that are not already in $\tau(Q)$. Retrieval stops when no new query terms are covered or when all query terms are covered. The original work uses $M=0.95$ and $T=2$ or $T=4$ depending on the dataset.

Adapted here as a reranker, AIR is applied to the FSR candidate set $\mathcal{C}_t$: the query Q is the question q, the reformulated query Q_r (with the expansion rule above) is built from the current evidence E_t, and each $d\in\mathcal{C}_t$ is ranked by $s(Q_r,d)$. Dividing by $|Q_r|$ would not change the ranking and is omitted. If $Q_r=\varnothing$ (all query terms are covered), AIR's stopping criterion is met and no reformulated query exists; in that case we keep the fused FSR ordering already defined for the no-reranking control, i.e. ranking $\mathcal{C}_t$ by $\max\left(\mathrm{FSR}(d,q),\max_i\mathrm{FSR}(d,g_i)\right)$. AIR does not use the gap G_t: its own notion of missing information is Q_r. Unlike GAR, AIR has no separate relevance, coverage, and novelty terms; coverage enters only through query reformulation.

3.10 Dartboard / RIG (Relevant Information Gain), Adapted

Dartboard selects a set of passages that maximizes the total relevant information gained, so that redundancy is penalized implicitly rather than through an explicit trade-off weight [Pickett et al., 2024]. It is modeled as a game: a target passage T is drawn with probability $P(T=t\mid q,\sigma)=\mathcal{N}(q,t,\sigma)$, the query lands near it, and a set of guesses G is scored by how close its closest guess is to the target. With a Gaussian kernel the objective (paper Eq. 2) is, up to sign and constants:

$$s(G,q,A,\sigma)\;\propto\;\sum_{t\in A}\mathcal{N}(q,t,\sigma)\,\max_{g\in G}\mathcal{N}(t,g,\sigma),$$

which is maximized, where A is the set of points considered (here the candidate set $\mathcal{C}_t$), σ is the standard deviation of the kernel, and $\mathcal{N}$ is computed from the distance between embeddings (cosine-based in the original).

The method is applied greedily. The first selected passage is the one closest to the query:

$$d_1=\arg\max_{d\in\mathcal{C}_t}\mathcal{N}(q,d,\sigma).$$

Each subsequent passage maximizes the score of the enlarged set:

$$S_{\mathrm{Dartboard}}(d\mid G)=\sum_{t\in\mathcal{C}_t}\mathcal{N}(q,t,\sigma)\,\max\Big(\max_{g\in G}\mathcal{N}(t,g,\sigma),\;\mathcal{N}(t,d,\sigma)\Big),\qquad d\in\mathcal{C}_t\setminus G,$$

and the reranked list is the order of selection. The original implementation works in log space with a log-sum-exp for numerical stability, which gives the same ranking. Because the maximum ensures that an exact duplicate adds no information, duplicates are not selected until all non-duplicates are exhausted. Only the scope is adapted: the original set-level greedy objective is applied unchanged to $\mathcal{C}_t$, and no simplified relevance-minus-similarity score is substituted for it. As in the original,

Dartboard uses neither the collected evidence E_t nor the gap G_t; it is included because it directly motivated GAR's novelty term N (Section 4). Its only hyperparameter is σ, and the original work shows that it generalizes both nearest-neighbor retrieval and MMR.

3.11 RankT5

RankT5 fine-tunes T5 so that it outputs a real-valued ranking score for each query–document pair instead of generating a text token [Zhuang et al., 2023]. Two structures are studied: an encoder–decoder model that uses the first output token of the decoder, and an encoder-only model that omits the decoder.

$$S_{\text{RankT5}}(q, d) = f_\theta(q, d), \qquad d \in \mathcal{C}_t,$$

where f_θ is fine-tuned with a ranking objective: pointwise cross-entropy, pairwise logistic, or listwise softmax cross-entropy. The pairwise and listwise losses use groups of candidates only during training; at inference each pair is scored independently. RankT5 represents the learning-to-rank family of rerankers.

3.12 RankGPT (LLM Listwise Reranking)

RankGPT prompts a large language model with the question and a group of passages, each labelled with an identifier ($[1], [2], \ldots$), and asks it to output the identifiers in descending order of relevance, for example $[2] > [3] > [1]$, without producing intermediate relevance scores [**?**].

$$(p_1, \ldots, p_w) \xrightarrow{\text{LLM permutation generation}} [\pi_1] > [\pi_2] > \cdots > [\pi_w].$$

Because of context-length limits, a candidate list of M passages is reranked with a sliding window of size w and step size s, from back to front: the passages ranked $M - w + 1$ to M (1-based positions, exactly w passages) are ranked first, then the window moves toward the top by s positions (the next window covers positions $M - w - s + 1$ to $M - s$, and so on), so the best passages of one window carry into the next. The original work uses $w = 20$, $s = 10$ on the top-100 first-stage passages; this top-100 setting is only the configuration of the original RankGPT paper, and in this experiment RankGPT is applied to the full common candidate set $\mathcal{C}_t$ using the same sliding-window procedure. The original work also reports that the result depends strongly on the initial order; here the initial order is the fused FSR order used for the common candidate set, i.e. $\mathcal{C}_t$ ranked by $\max\left(\text{FSR}(d, q), \max_i \text{FSR}(d, g_i)\right)$. If the model omits identifiers, the missing passages are appended in their original order. RankGPT is included because MIGRES is itself LLM-driven, making an LLM-based reranking baseline a natural comparison point.

3.13 Reranking Baseline Comparison

System	Reranking type	Role
MIGRES+FSR	No reranking	Control
MIGRES+FSR+MonoBERT	Cross-encoder pointwise	Reranking baseline
MIGRES+FSR+MonoT5	Seq2seq pointwise	Reranking baseline
MIGRES+FSR+MonoT5-G	Seq2seq pointwise, query+gap	Gap-aware control baseline
MIGRES+FSR+DuoBERT	Pairwise	Reranking baseline
MIGRES+FSR+ColBERTv2	Late interaction	Reranking baseline
MIGRES+FSR+LambdaMART	Classical learning-to-rank	Reranking baseline
MIGRES+FSR+RankT5	Neural learning-to-rank	Reranking baseline
MIGRES+FSR+MMR	Relevance vs. redundancy	Closest conceptual baseline
MIGRES+FSR+AIR	Gap-coverage-driven	Adapted baseline; precursor to C
MIGRES+FSR+Dartboard/RIG	Relevance vs. novelty	Adapted baseline; precursor to N
MIGRES+FSR+RankGPT	LLM listwise	Reranking baseline
MIGRES+FSR+GAR	Gap-aware reranking	Proposed method

All rerankers are applied to the identical FSR candidate set $\mathcal{C}_t$ for each question, so that only the reranking method varies across systems. AIR and Dartboard/RIG are included not merely as baselines but as the two prior works that directly motivated GAR's coverage and novelty terms; the comparison therefore also tests whether GAR's fused three-signal score outperforms each of its own inspirations used alone.

3.14 Baseline Configuration Protocol

All rerankers are configured under one rule, fixed before any D_{test} result is inspected, so that the comparison with GAR is controlled.

- **Adaptations.** Two baselines are *adapted* rather than reproduced. AIR is turned from an iterative query-reformulation method into a single-pass reranker of $\mathcal{C}_t$ (Section 3.9), and RankGPT is applied to the full common candidate set with the sliding window of Section 3.12. Dartboard/RIG keeps its original set-level greedy objective and is adapted only in scope (it selects from $\mathcal{C}_t$). Each adaptation is recorded in the configuration, and these three systems are described as adapted baselines in all reporting.
- **Checkpoints.** Every neural baseline uses a published checkpoint, identified by exact name and version, without fine-tuning on any D_{val} or D_{test} data. If no public checkpoint exists for a method (possibly RankT5), it is trained following the original recipe on the original training data (MS MARCO), never on the evaluation datasets.
- **Hyperparameters.** Each baseline's free hyperparameters (λ_{MMR}; σ for Dartboard; M and T for AIR; the tree parameters of LambdaMART) are either fixed at the values of the original publication or tuned on D_{val} only, with Recall@k as the objective, over a grid fixed and recorded in advance. The number of configurations tried for each system is recorded. GAR uses Bayesian optimization, so the search budgets are not identical, and this is reported as a limitation together with the shared-K_0 limitation of Section 6.13.5.
- **LambdaMART.** It is implemented with a gradient-boosted-tree library using a LambdaRank objective (for example LightGBM). Its features are fixed in advance: the fused first-stage score, the first-stage rank, the fraction of $\tau(q)$ found in $\tau(d)$, and document length in tokens. It is trained on the official training split using question-only first-stage candidate lists from the selected FSR with binary relevance labels from the evidence annotations, because no MIGRES gap trajectories are generated on the training split. Its tree hyperparameters are tuned on D_{val}.

- **Gap-aware neural control.** MonoT5-G is the same MonoT5 checkpoint and scoring rule as MonoT5, but it scores each candidate against the original question and each current gap requirement and uses

$$S_{\text{MonoT5-G}}(d) = \max\left(S_{\text{MonoT5}}(q, d), \max_i S_{\text{MonoT5}}(g_i, d)\right).$$

  This isolates the effect of giving a strong pointwise reranker access to the same gap information available to GAR; it uses no GAR coverage or novelty term.

- **RankGPT.** One fixed language model is used, namely the answer-generation model of the MIGRES configuration, recorded with its exact version. Decoding is deterministic (temperature 0), the permutation-generation prompt of **?** is used and stored verbatim, the window is $w = 20$ with step $s = 10$, and the initial order is the fused first-stage order of Section 6.4.

- **Input length.** Each neural baseline truncates the document tail at its own maximum input length; the fraction of truncated candidates is reported for every baseline.

Any training or tuning above uses no test annotations, and all resulting settings are frozen before D_{test} is run.

4 Development and Introduction of GAR

4.1 What GAR Is

GAR is a reranking algorithm designed for iterative retrieval-augmented generation. It reranks the candidate set $\mathcal{C}_t$ returned by FSR's first-stage retrieval, rather than searching the corpus $\mathcal{D}$ directly. It changes the ranking question from:

"Is this document relevant to the original question?"

to:

"Is this document useful given what is missing and what is already known?"

GAR uses three signals:

1. **Relevance**: relevance to the original question.
2. **Gap coverage**: how much of the missing information is covered by the candidate.
3. **Evidence novelty**: how much new information the candidate adds beyond existing evidence.

4.2 Why GAR Was Developed

A query-only retriever can repeatedly return documents that are strongly related to the question but do not resolve the current missing information. MIGRES already exposes the missing-information state, making it possible for the retrieval algorithm to use that state directly [Wang et al., 2025].

The motivation for the GAR components comes from three sources.

First, MIGRES provides the missing-information concept: later retrieval can be targeted toward information that is still missing [Wang et al., 2025].

Second, Yadav et al. use an iterative evidence-retrieval process that focuses later retrieval on information not covered by existing justifications. This motivates the coverage component in GAR [Yadav et al., 2020].

Third, Relevant Information Gain argues that retrieval should avoid wasting limited context on redundant information while retaining relevance [Pickett et al., 2024]. This motivates the novelty component.

GAR combines these ideas into one retrieval score. The formulation is therefore a proposed unified model; the project does not claim that each component is independently novel.

4.3 Applications

GAR is intended for settings where information needs change during retrieval, including:

- multi-hop question answering;
- iterative RAG;
- research-oriented evidence retrieval;
- evidence gathering over large document collections;
- systems that explicitly track missing information.

GAR does not require a new language model. The surrounding system can provide the current gap and evidence state.

5 Mathematical Formulation

5.1 Notation

At retrieval iteration t:

$$q = \text{original question},$$

$$E_t = \text{evidence collected before iteration } t,$$

$$G_t = \{g_1, \ldots, g_m\}$$

is the set of current information requirements, and

$$\mathcal{D} = \text{searchable document corpus.}$$

The first-stage retriever performs retrieval over $\mathcal{D}$, using K_0 as the retrieval depth for each individual query (the question or a single gap requirement), and returns a candidate set:

$$\mathcal{C}_t \subseteq \mathcal{D}, \qquad |\mathcal{C}_t| \leq (m+1)K_0.$$

As Eq. (9) makes explicit, $\mathcal{C}_t$ is the deduplicated union of $m+1$ separate top-K_0 retrievals (one for q and one for each $g_i \in G_t$), so its size is bounded above by $(m+1)K_0$ rather than equal to K_0; K_0 should be read as the per-query retrieval depth, not the final candidate-pool size.

A candidate document is a member of this candidate set, not of the full corpus directly:

$$d \in \mathcal{C}_t.$$

An individual evidence item is:

$$e \in E_t.$$

The GAR input is therefore:

$$\boxed{(q, E_t, G_t, \mathcal{C}_t)}.$$

GAR never receives $\mathcal{D}$ directly; $\mathcal{D}$ is used only by the first-stage retriever to construct $\mathcal{C}_t$ (Section 6). Which concrete algorithm performs this first-stage retrieval is itself an empirical choice, resolved by the retriever comparison in Section 2 before the reranking comparison in Section 3 and Section 8 is run; once selected, that retriever is held fixed so that only the reranking method varies across the main comparison.

5.2 Text Embeddings

GAR needs a fixed representation function:

$$\phi : \mathcal{T} \to \mathbb{R}^p.$$

We use:

`sentence-transformers/multi-qa-mpnet-base-cos-v1`

as the fixed embedding model. The model maps sentences and paragraphs to 768-dimensional dense vectors and was designed for semantic search [Reimers and Gurevych, 2019, Sentence Transformers, 2020].

The same model is used to embed the question, candidate documents, and collected evidence, for the relevance and novelty terms; gap requirements are not embedded and are processed by the token-overlap Coverage term. The model accepts at most 512 word pieces, so any question, evidence item, or candidate longer than this is truncated to its first 512 word pieces before embedding, and no chunking or pooling over chunks is used; the fraction of truncated inputs is reported per dataset. The corpus passage is the retrieval unit, so most candidates are expected to fall within this limit. Candidate document embeddings are computed on demand for $\mathcal{C}_t$; GAR does not maintain a corpus-wide embedding index, since the first-stage retriever operates directly on raw text.

5.3 Cosine Similarity

For two text items x and y:

$$\cos(x, y) = \frac{\phi(x)^T \phi(y)}{\|\phi(x)\|_2 \|\phi(y)\|_2}.$$

For a normalized score in $[0, 1]$, GAR uses:

$$\boxed{s(x, y) = \frac{1}{2}\left(1 + \frac{\phi(x)^T \phi(y)}{\|\phi(x)\|_2 \|\phi(y)\|_2}\right)} \tag{1}$$

The embedding model creates vectors; cosine similarity compares them. This formula assumes both $\phi(x)$ and $\phi(y)$ are nonzero, which holds in practice for the embedding model used here.

5.4 Relevance

Relevance measures how well a candidate answers the original question, independent of what is already known or still missing:

$$\boxed{R(d, q) = s(d, q)} \tag{2}$$

Relevance is deliberately simple: it reuses the single similarity primitive from Eq. (1) with no additional blending or hyperparameter. Whether a candidate also addresses the current information gap is a separate question, answered by the Coverage term below — keeping the two concerns apart avoids mixing two different notions of "relevant" into one number.

5.5 Gap Coverage

Let $\tau(x)$ be the set of normalized content tokens obtained after the fixed preprocessing pipeline. The pipeline is: Unicode NFKC normalization; lowercasing; tokenization into alphanumeric word tokens (the regular expression `\w+`); removal of English stopwords (the NLTK English list); and Porter stemming; the result is treated as a set. The same pipeline is used everywhere $\tau(\cdot)$ appears (including the baselines of Sections 3.8–3.9 and the LambdaMART term-overlap feature), it is applied to the full text of d as retrieved (not to a truncated version), and the exact library versions are recorded.

For a non-empty gap requirement:

$$\boxed{c(d, g_i) = \frac{|\tau(d) \cap \tau(g_i)|}{|\tau(g_i)|}} \tag{3}$$

This is normalized token recall. The coverage score is the average recall across all gap requirements:

$$\boxed{C(d, G_t) = \frac{1}{m} \sum_{i=1}^{m} c(d, g_i)} \tag{4}$$

Thus $C(d, G_t) \in [0, 1]$.

If $\tau(g_i)$ is empty, that invalid requirement is removed before retrieval. Eq. (4) requires $m > 0$; if $G_t = \varnothing$ after removing invalid requirements, MIGRES has determined that no information is currently missing, and the iterative retrieval loop stops for that question rather than evaluating C on an empty gap set.

5.6 Evidence Novelty

For non-empty evidence:

$$\boxed{N(d, E_t) = 1 - \max_{e \in E_t} s(d, e)} \tag{5}$$

A candidate very similar to existing evidence receives low novelty. A candidate dissimilar to all existing evidence receives high novelty.

At the first iteration:

$$E_0 = \emptyset.$$

We define:

$$\boxed{N(d, \emptyset) = 1.} \tag{6}$$

5.7 Final GAR Score

The final score is:

$$\boxed{S(d \mid q, E_t, G_t) = \alpha R(d, q) + \beta C(d, G_t) + \gamma N(d, E_t)} \tag{7}$$

subject to:

$$\boxed{\alpha, \beta, \gamma \geq 0, \qquad \alpha + \beta + \gamma = 1.} \tag{8}$$

Because the three components are normalized, the final score is also in $[0, 1]$.

5.8 Hyperparameters

GAR has three scoring hyperparameters:

$$\boxed{\alpha, \beta, \gamma}$$

and one retrieval hyperparameter:

$$\boxed{K_0}.$$

Their roles are:

Parameter	Role	Constraint
α	Weight of relevance R	≥ 0
β	Weight of coverage C	≥ 0
γ	Weight of novelty N	≥ 0
K_0	First-stage retrieval depth per query (question or gap requirement); candidate pool $\lvert\mathcal{C}_t\rvert \leq (m+1)K_0$	$K_0 > 0$
k	User-configurable final retrieval depth returned to MIGRES; fixed across systems within a comparison	$k \in \mathbb{Z}_{\geq 1}$

The final retrieval depth k is an **user-configurable experimental parameter**, not a GAR-specific scoring parameter. The implementation accepts k from the experiment configuration (for example, a command-line argument or configuration file); no particular default value is required by the method. Once supplied, k must be a positive integer and is kept identical across all systems and rerankers within a comparison.

Two depths must not be confused. K_0^{sel} is a fixed, pre-specified per-query depth used only to select FSR in E1 (Section 8.2). K_0 is the per-query depth that Bayesian optimization tunes for GAR (Section 6.13.4), and it is tuned only after FSR has been selected and frozen, so the selection of FSR does not depend on the value optimization chooses. Neither depth implies a post-union truncation:

$\mathcal{C}_t$ is the complete deduplicated union of Eq. (9), so its size varies with m and with the overlap between the $m + 1$ lists, and the mean and maximum $|\mathcal{C}_t|$ are reported for every dataset.

The fixed embedding model and cosine similarity are also configuration choices, not learned GAR hyperparameters.

6 How GAR Works Using MIGRES

6.1 Overall Architecture

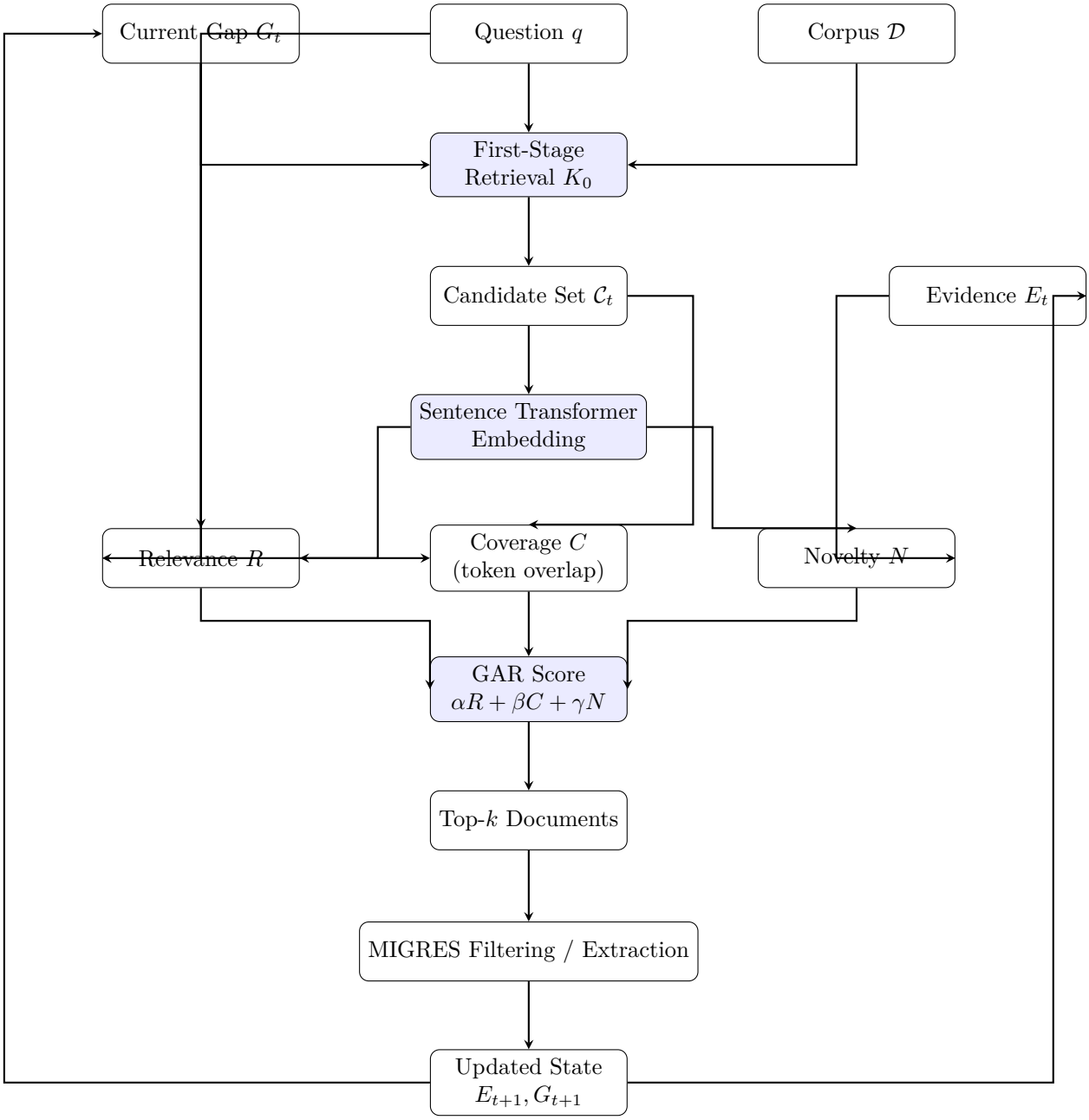


Figure 1: Overall architecture of GAR within the MIGRES framework. MIGRES provides the current question, evidence, and information gap. The selected first-stage retriever (Section 2) retrieves over the corpus $\mathcal{D}$ to produce the candidate set $\mathcal{C}_t$. Relevance and Novelty are computed from Sentence Transformer embeddings of the candidates, the question q (for Relevance), and the collected evidence E_t (for Novelty); Coverage is computed directly from candidate and gap text via token overlap and does not require embeddings. The reranked documents are then returned to MIGRES for filtering, extraction, and state updating.

6.2 Step 1: Dataset Question

A benchmark supplies the original question:

$$q.$$

The benchmark's gold answer and supporting evidence are not given to GAR during retrieval.

6.3 Step 2: MIGRES Creates the Retrieval State

MIGRES processes the question and maintains:

$$E_t = \text{evidence already collected}$$

and:

$$G_t = \text{information still missing.}$$

The gap can contain multiple requirements:

$$G_t = \{g_1, \ldots, g_m\}.$$

GAR does not independently invent G_t in the main experiment. It receives the gap produced by MIGRES.

6.4 Step 3: First-Stage Retrieval

The selected first-stage retriever (Section 2) searches the corpus $\mathcal{D}$ using its own retrieval representation and index; for a sparse lexical retriever such as BM25 this is a lexical index, while dense or late-interaction retrievers use their corresponding pre-built corpus representations. For each query-like input x, it returns the K_0 highest-scoring documents:

$$\mathrm{FSR}_{K_0}(x, \mathcal{D}).$$

GAR's candidate set is built using both the original question and every gap requirement, so that first-stage retrieval is itself gap-aware:

$$\boxed{\mathcal{C}_t = \mathrm{FSR}_{K_0}(q, \mathcal{D}) \cup \bigcup_{i=1}^{m} \mathrm{FSR}_{K_0}(g_i, \mathcal{D})} \tag{9}$$

Duplicates are removed. Because each of the $m+1$ retrieval calls can return up to K_0 documents, the resulting candidate pool satisfies $|\mathcal{C}_t| \leq (m+1)K_0$ after deduplication, not $|\mathcal{C}_t| = K_0$; K_0 is the retrieval depth used for each individual query, while k is the final number of documents returned after reranking.

Taking the union in Eq. (9) yields a set in which a document can appear in more than one of the $m+1$ retrieved lists with different first-stage scores under q or a given g_i. These individual FSR scores are retained alongside $\mathcal{C}_t$ for the baselines that need them (LambdaMART's first-stage-score feature, whose value is the fused score defined below, and the fused ordering used by AIR and RankGPT), but they are not used in GAR's own score. GAR itself does not need these first-stage scores, as

it rescoring every $d \in \mathcal{C}_t$ from scratch using R, C, and N. However, the no-reranking control (the selected first-stage retriever with no reranker) does need a defined top-k over $\mathcal{C}_t$: for that system, we fuse a document's score as the maximum first-stage score it received across the retrieval calls that returned it, i.e. $\max\left(\mathrm{FSR}(d,q), \max_i \mathrm{FSR}(d,g_i)\right)$, and rank $\mathcal{C}_t$ by this fused score.
GAR itself does not build or query a corpus-wide dense embedding index; any indexing cost required by the selected FSR is part of FSR and is measured separately in E5.

6.5 Step 4: Convert Candidate Text to Embeddings

GAR uses the fixed Sentence Transformer model:

$$\phi(x).$$

The question, each evidence item, and *only the documents already in* $\mathcal{C}_t$ are represented as vectors for the relevance and novelty terms; gap requirements are not embedded and are processed by the token-overlap Coverage term. Because $|\mathcal{C}_t| \leq (m+1)K_0 \ll |\mathcal{D}|$, GAR never needs to embed or index the full corpus — embeddings are computed on demand for the small candidate set returned by the first-stage retriever.

6.6 Step 5: Calculate Relevance

For every candidate:

$$R(d,q)$$

measures:

question-document relevance.

6.7 Step 6: Calculate Coverage

GAR then asks:

"Does this document cover the information that is missing?"

This produces:

$$C(d, G_t).$$

6.8 Step 7: Calculate Novelty

GAR then asks:

"Is this document redundant with evidence already collected?"

This produces:

$$N(d, E_t).$$

6.9 Step 8: Calculate the GAR Score

For every candidate:

$$S(d \mid q, E_t, G_t) = \alpha R + \beta C + \gamma N.$$

The candidate with the highest score is preferred because it is simultaneously relevant, gap-covering, and non-redundant.

6.10 Step 9: Return Top-k

GAR returns:

$$\boxed{T_t = \mathrm{Top}_k\{d \in \mathcal{C}_t : \ S(d \mid q, E_t, G_t)\}} \tag{10}$$

where k is fixed for the experiment. Because $\mathcal{C}_t$ contains the question retrieval $\mathrm{FSR}_{K_0}(q, \mathcal{D})$, $K_0 \geq k$ is sufficient to guarantee $|\mathcal{C}_t| \geq k$ when each retrieval returns K_0 documents, but it is not necessary: with $K_0 < k$, the union of the question and gap retrievals can still contain at least k unique candidates. When fewer than k unique candidates exist, GAR returns all $\min(k, |\mathcal{C}_t|)$ of them, ranked by S.

6.11 Step 10: MIGRES Continues

MIGRES receives the retrieved documents and performs its normal filtering and extraction.
It updates:

$$E_{t+1}$$

and, if more information is needed:

$$G_{t+1}.$$

GAR is then called again.
The complete loop is:

$$\boxed{(q, E_t, G_t, \mathcal{D}) \xrightarrow{\text{FSR}} \mathcal{C}_t \xrightarrow{\text{GAR}} T_t \xrightarrow{\text{MIGRES}} (E_{t+1}, G_{t+1}) \xrightarrow{\text{FSR}} \mathcal{C}_{t+1} \xrightarrow{\text{GAR}} T_{t+1}}$$

This is why GAR is evaluated inside MIGRES rather than as an isolated one-shot reranker.

6.12 Retrieval vs. Reranking in This Design

The two stages must not be confused:

$$\boxed{\text{First-stage retrieval} \rightarrow \text{Embedding} \rightarrow \text{GAR scoring} \rightarrow \text{Top-}k}$$

The first-stage retriever finds a manageable candidate set from the full corpus $\mathcal{D}$.
Embedding converts only that candidate set (plus the question q and the evidence E_t) into vectors; gap requirements G_t are not embedded and are handled by token-overlap Coverage.
GAR makes the final ranking decision over the candidate set.
This is precisely the reranking design proposed in this project: a first-stage retriever narrows the corpus down to $\mathcal{C}_t$, and GAR reranks that output using relevance, gap coverage, and evidence novelty.
A version of GAR that performs its own first-stage candidate generation directly from $\mathcal{D}$ (for example, via an ANN index over GAR's own embeddings, without relying on a separate first-stage retriever) would instead be a complete end-to-end retrieval algorithm; that variant is not evaluated here and is left as a direction for future work.

6.13 Development, Validation, and Test Protocol

6.13.1 Is GAR Trained?

No.
The proposed GAR algorithm has no trainable parameters. We do not train GAR on D_{train}.
The embedding model is a fixed pretrained Sentence Transformer. It is not fine-tuned in the primary experiment.
Therefore the experiment has no GAR training stage.

6.13.2 Training Data

If a baseline uses an official pretrained checkpoint, we use that checkpoint rather than retraining it on the evaluation data.
The benchmark training split may be used for auxiliary development or baseline-specific training only if required by the official implementation. Any such use must be documented, must not use test annotations, and follows the protocol of Section 3.14.
For GAR itself:

$$D_{\text{train}} \not\to \text{GAR parameter learning.}$$

6.13.3 Validation Set D_{val}

We define:

$$\boxed{D_{\text{val}} = \text{the held-out development/validation split.}}$$

Where an official development split exists, we use it; where the official test annotations are withheld, the development split is divided into a validation half and a test half as described in Section 7.10. The exact files and split names are recorded for each dataset.
GAR hyperparameters are selected using the *complete MIGRES+FSR+GAR pipeline* on D_{val}.
This is important because GAR cannot be meaningfully validated in isolation: E_t and G_t are produced by the MIGRES iterative process, and $\mathcal{C}_t$ is produced by FSR.
The validation loop is:

$$D_{\text{val}} \to \text{MIGRES} \to (q, E_t, G_t) \to \text{FSR} \to \mathcal{C}_t \to \text{GAR} \to \text{MIGRES} \to \text{validation metric.}$$

6.13.4 Bayesian Optimization

We use Bayesian optimization to select:

$$\theta_{\text{GAR}} = \{\alpha, \beta, \gamma, K_0\}.$$

Bayesian optimization uses previous validation evaluations to decide which configuration to test next rather than exhaustively evaluating every possible combination [Snoek et al., 2012].
The search space is:

$$\alpha, \beta, \gamma \geq 0, \qquad \alpha + \beta + \gamma = 1,$$

and a predefined discrete candidate set for K_0, for example:

$$K_0 \in \{50, 100, 200, 500\}.$$

The exact search space and random seed are recorded before the experiment.
The optimization objective must be selected before test evaluation. The primary objective should be a retrieval metric aligned with the main research question, such as Recall@k on D_{val}.
The procedure is:

$$\boxed{D_{\text{val}} \to \text{MIGRES+FSR+GAR} \to \text{validation score} \to \text{Bayesian optimization} \to \theta^*_{\text{GAR}}}$$

The selected configuration is then frozen.

6.13.5 Test Set D_{test}

After validation:

$$\boxed{\theta^*_{\text{GAR}} \text{ is frozen.}}$$

We then evaluate:

$$D_{\text{test}} \rightarrow \text{MIGRES+FSR+GAR}(\theta^*_{\text{GAR}}) \rightarrow \text{final metrics.}$$

No test result is used to change the hyperparameters.
The same k, dataset, corpus, MIGRES configuration, and stopping conditions are used for the comparison systems wherever possible.
Because K_0 is one of the parameters selected by Bayesian optimization on D_{val} for GAR specifically (Section 6.13.4), and the same frozen K_0 then fixes the candidate pool $\mathcal{C}_t$ passed to every baseline reranker, the candidate-pool depth is not independently tuned for each baseline. This is a limitation of the comparison: it should be read as GAR vs. baselines under a candidate-pool depth chosen to favor GAR's own validation performance, not under a depth optimized separately for each system. A stricter design would either fix K_0 in advance independently of any reranker's performance, or tune each baseline's own hyperparameters (including its effective candidate depth, where applicable) on D_{val} under the same protocol used for GAR.

6.13.6 Complete Experimental Flow

1. Prepare the official dataset splits and searchable corpus.
2. Run the first-stage-retriever comparison (Section 2) on D_{val} and select FSR as the best-performing candidate; freeze FSR for every step below.
3. Reproduce MIGRES+FSR as the no-reranking control.
4. Verify the MIGRES retrieval interface and confirm how E_t and G_t are exposed.
5. Implement on-demand embedding of FSR's candidate set $\mathcal{C}_t$ using the fixed Sentence Transformer.
6. Implement and unit-test the GAR equations.
7. Run Bayesian optimization on D_{val} through the complete MIGRES+FSR+GAR pipeline.
8. Freeze $\alpha, \beta, \gamma, K_0$.
9. Run the held-out D_{test} with the frozen configuration.
10. Run MonoBERT, MonoT5, MonoT5-G, DuoBERT, ColBERTv2, LambdaMART, RankT5, MMR, AIR, Dartboard/RIG, and RankGPT as rerankers on the identical FSR candidate set, using their established implementations.
11. Evaluate retrieval quality, downstream answer quality, efficiency, and cost.
12. Run GAR ablations using the same frozen evaluation protocol.

7 Datasets

7.1 Dataset Selection

The project uses the six datasets reported in the MIGRES evaluation:

WikiHop, HotpotQA, MuSiQue, Natural Questions, TriviaQA, StrategyQA

These datasets provide questions and task-specific answer/evidence annotations suitable for evaluating retrieval and downstream question answering.

7.2 WikiHop

Paper: Welbl et al. [2018].
WikiHop is a multi-document, multi-hop reading-comprehension benchmark designed so that the answer requires combining information across documents. The dataset is useful for GAR because it tests whether retrieval can assemble complementary evidence rather than repeatedly retrieving one topic.
Typical fields include a question, candidate documents, an answer, and supporting information.
Paper: https://aclanthology.org/Q18-1021/

7.3 HotpotQA

Paper: Yang et al. [2018].
HotpotQA contains about 113K Wikipedia-based question-answer pairs and provides sentence-level supporting facts. It includes multi-hop and comparison questions [Yang et al., 2018].
For first-stage development, we use the FullWiki setting because it requires retrieval from a larger Wikipedia corpus. The official repository provides train, development, and FullWiki resources and explicitly allows researchers to replace its retrieval component with their own IR system.
Paper: https://aclanthology.org/D18-1259/
Official resources: https://hotpotqa.github.io/
The repository documentation describes the FullWiki development and test files and the use of custom IR systems.

7.4 MuSiQue

Paper: Trivedi et al. [2022].
MuSiQue was designed to reduce shortcut solutions in multi-hop QA. MuSiQue-Ans contains about 25K questions requiring 2–4 hops [Trivedi et al., 2022].
Typical information includes a question, answer, reasoning structure, and supporting evidence.
Paper: https://aclanthology.org/2022.tacl-1.31/
Official repository: https://github.com/StonyBrookNLP/musique

7.5 Natural Questions

Paper: Kwiatkowski et al. [2019].
Natural Questions consists of real anonymized queries issued to Google Search and Wikipedia-based annotations. The public release contains about 307K training examples, 7,830 development examples, and 7,842 sequestered test examples [Kwiatkowski et al., 2019]. The exact training count is reported as 307,373 in the paper and in the data-statistics section of the official repository, while the repository's data-description paragraph gives 307,372; the count of the downloaded release is recorded in the experiment configuration.
Typical fields include a question, Wikipedia page information, long-answer annotations, short-answer annotations, and document-level information.
Paper: https://aclanthology.org/Q19-1026/
Official research page: https://ai.google/research/NaturalQuestions

7.6 TriviaQA

Paper: Joshi et al. [2017].
TriviaQA contains about 95K question-answer pairs and more than 650K question-answer-evidence triples. Questions were written by trivia enthusiasts and paired with independently gathered evidence documents [Joshi et al., 2017].
Typical fields include a question, answer, aliases, and evidence documents.
Paper: https://aclanthology.org/P17-1147/
Official resources: https://nlp.cs.washington.edu/triviaqa/

7.7 StrategyQA

Paper: Geva et al. [2021].
StrategyQA contains 2,780 questions with implicit reasoning strategies. Each example includes a question, a reasoning decomposition, and Wikipedia evidence paragraphs [Geva et al., 2021].
This makes it useful for testing whether gap-aware reranking helps when the required reasoning steps are not directly stated in the question.
Paper: https://aclanthology.org/2021.tacl-1.21/
Official resources: https://allenai.org/data/strategyqa

7.8 Dataset Summary

Dataset	Scale	Useful properties for GAR
WikiHop	∼51K	Multi-document, multi-hop reasoning.
HotpotQA	∼113K	Multi-hop QA with sentence-level supporting facts; FullWiki retrieval.
MuSiQue	∼25K	2–4 hop compositional reasoning.
Natural Questions	∼307K train; 7,830 dev; 7,842 test (withheld)	Real search questions with Wikipedia annotations.
TriviaQA	∼95K QA pairs; >650K evidence triples	Heterogeneous evidence and lexical variation.
StrategyQA	2,780	Implicit reasoning with evidence paragraphs.

Exact counts used in experiments must be taken from the downloaded release and recorded in the experiment configuration. Dataset counts can differ by release or preprocessing.

7.9 What a Dataset Example Looks Like

The common conceptual structure is:

$$\boxed{\text{Question} + \text{Corpus/Evidence} + \text{Answer} + \text{Ground-Truth Evidence/Annotations}}$$

For example, a HotpotQA-style example contains a natural-language question, an answer, supporting facts, and Wikipedia context. During reranking, GAR receives the question, MIGRES's evidence and gap state, and the candidate set already retrieved by FSR from the searchable corpus. The gold answer and gold supporting evidence are reserved for evaluation.
The ground-truth evidence must never be used to construct G_t, E_t, or the FSR candidate pool during test-time retrieval.

7.10 Dataset Split Policy

For each dataset:

$$D_{\text{train}}, \quad D_{\text{val}}, \quad D_{\text{test}}$$

are treated separately where official splits are available.
For GAR:

$$D_{\text{val}}$$

is used for hyperparameter optimization, while:

$$D_{\text{test}}$$

is used only for the final evaluation.

Labelled test data. The retrieval metrics of this project (Recall, Precision, MRR, nDCG, GapEvidenceCoverage, and CandRecall) need ground-truth evidence annotations, and these are withheld for some official test sets. For Natural Questions the 7,842 test examples are withheld and can only be scored through the official competition site, and the HotpotQA FullWiki test set is released without answers or supporting facts. This is checked for every dataset and recorded in the experiment configuration. For any dataset whose official test annotations are unavailable, D_{val} and D_{test} are two disjoint, question-level halves of the labelled official development split, divided once with a recorded seed before any experiment is run. Neither half is used to train or tune any system except D_{val} for the selections described in Section 8.2 and Section 6.13, and the official training split may be used only for baseline-specific training under Section 3.14. All retrieval metrics and all confirmatory tests are computed on this labelled D_{test}.
Official hidden-test evaluation is optional. If it is performed, it yields only the end-to-end answer metrics returned by the official evaluation procedure (no retrieval metrics), is run once per system with the frozen configuration, and is reported separately rather than pooled with the labelled D_{test} analyses.

8 Experimental Design and Analysis

This section is the complete experimental protocol of the project. It fixes, before any D_{test} result is inspected, which systems are compared, which research questions and hypotheses are tested, which metrics are computed, how the ablations are run, and how the statistics are carried out. It does not modify the GAR formulation of Sections 5 and 6 (Eqs. (1)–(10)), the MIGRES setup of Section 1, or the datasets of Section 7, and it reports no results: every statement below about performance is a question or a hypothesis, not a finding.
Throughout, FSR is the first-stage retriever selected in Experiment E1, $\mathcal{C}_t$ is the candidate set of Eq. (9), K_0 is the per-query retrieval depth, k is the final retrieval depth returned to MIGRES, and D_{val}, D_{test} are the validation and test splits of the six datasets of Section 7. All experiments are run separately on each of the six datasets; "across datasets" always means the macro-average over the six.

8.1 Experimental Objectives

The experimental study has four purposes:

1. to compare first-stage information retrieval algorithms;
2. to compare GAR with established reranking algorithms on the same candidate set;
3. to determine which GAR components contribute to performance, through ablation;
4. to measure downstream question-answering quality, computational cost, and robustness (across datasets and across candidate-pool depths).

These purposes correspond to two different kinds of experiment, which are never mixed:

	First-stage retrieval (E1)	**Reranking (E2–E5)**
Input	The question q and the stored gap requirements $G_t = \{g_1, \ldots, g_m\}$ from the reference trajectory; the full corpus $\mathcal{D}$.	The fixed candidate set $\mathcal{C}_t$ built by the frozen FSR (Eq. (9)), together with q, E_t, G_t.
Output	A ranked list over $\mathcal{D}$ of depth K_0.	A reordering of $\mathcal{C}_t$; no document outside $\mathcal{C}_t$ can be returned.
Systems	BM25, DPR, Contriever, ColBERTv2, SPLADE.	FSR (no reranking), ten rerankers, GAR.
GAR present?	No.	Yes.
Purpose	Select FSR on D_{val}.	Test GAR, its components, its sensitivity to K_0, and its cost.

ColBERTv2 appears in both roles with different configurations: in E1 it searches the corpus $\mathcal{D}$ as a first-stage retriever; in E2 it only rescores the documents already in $\mathcal{C}_t$, as described in Section 3.

8.2 Experiment E1: First-Stage Information Retrieval Comparison

Systems.

BM25, DPR, Contriever, ColBERTv2, SPLADE.

These methods are compared *only* as candidate-generation (first-stage) retrieval systems, using their published or official implementations and checkpoints where possible (Section 2).
Protocol. All five systems are evaluated under the same conditions:

- the same dataset split (D_{val} of each dataset);
- the same questions;
- the same searchable corpus $\mathcal{D}$;
- the same per-query retrieval depth: every system returns its top 500 documents for each query, the largest value of the grid of Section 8.5, so that every shallower depth $\{50, 100, 200\}$ is obtained by truncating the same ranked list. The depth used for *selection* is fixed in advance as $K_0^{\text{sel}} = 100$ (the grid value nearest the middle of the range); it is not tuned on any result, and it is distinct from the K_0 later tuned for GAR (Section 5.8);
- GAR is not used in this experiment, and no MIGRES reranking is applied;
- gold answers and gold supporting evidence are never used during retrieval; they are used only to score the returned lists.

Candidate construction and reference trajectory. E1 follows the same candidate-construction procedure as the main experiment. For every D_{val} question, MIGRES with its original retriever (BM25) and no reranking is run once, and (q, E_t, G_t) is stored for every iteration t; no LLM call is repeated per candidate retriever. Each candidate retriever ρ then builds

$$\mathcal{C}_t^{\rho} = \rho_{K_0}(q, \mathcal{D}) \cup \bigcup_{i=1}^{m} \rho_{K_0}(g_i, \mathcal{D})$$

(Eq. (9)) from these stored queries, with duplicates removed and no further truncation. Because the stored gaps come from a trajectory produced with BM25 retrieval, this design may favour lexical retrievers slightly; the reference trajectory is recorded so this can be inspected. E1 also reports the question-only ranking $\rho_{K_0}(q, \mathcal{D})$ and the mean and maximum pool size $|\mathcal{C}_t^{\rho}|$ as diagnostics; only the union measurement is used for selection.
Retrieval-quality metrics. The selection metric is CandRecall@K_0^{sel}. For reference, the question-only ranking of every system is also scored with Recall@k; MRR@10; nDCG@10; and Precision@k, wherever the dataset annotations support binary relevance. Recall@k is reported at the final depth k and at each depth in $\{50, 100, 200, 500\}$. Definitions are given in Section 8.9.3.
Cost measurements. For each system we also report retrieval latency per query, indexing time, index memory, and the number of retrieved candidates (K_0^{E1}), on the same recorded hardware.
Selection rule. The primary purpose of E1 is to select FSR on D_{val}. The system with the highest macro-averaged (across the six datasets) candidate-pool recall CandRecall@K_0^{sel} on D_{val} (Section 8.9.3) is selected; it is chosen because candidate generation must not lose the evidence that later stages need, and it measures exactly the candidate set $\mathcal{C}_t$ that every reranker will receive. Because a larger pool trivially raises recall, the mean pool size is reported next to it. If two systems tie at the reported candidate recall, the tie is broken by smaller mean pool size, and then by lower latency. The selected system is frozen as FSR and used unchanged in E2–E5.
What E1 does and does not show. E1 does *not* test GAR and is not evidence that GAR is effective. It only determines the fixed retrieval substrate on which the reranking study is run.

8.3 Experiment E2: Main Reranking Comparison

Candidate set. After FSR is selected, the candidate set $\mathcal{C}_t$ is constructed once per question and iteration by Eq. (9), with the validation-selected K_0 (Section 6.13.5). **Every reranker receives the identical candidate set $\mathcal{C}_t$ for the same question and iteration.** The only intended experimental variable is the reranking method.
Systems.

FSR (no reranking), MonoBERT, MonoT5, MonoT5-G, DuoBERT, ColBERTv2, LambdaMART, RankT5, MMR, AIR, Dartboard/RIG, RankGPT, GAR.

The no-reranking control ranks $\mathcal{C}_t$ by the fused score $\max\left(\mathrm{FSR}(d,q), \max_i \mathrm{FSR}(d,g_i)\right)$ (Section 6.4). Each reranker is used in the form described in Section 3, including the adaptations of AIR and RankGPT to the common candidate set. Every reranker outputs a complete ordering of $\mathcal{C}_t$ (the greedy methods MMR, AIR, and Dartboard/RIG continue their selection until $\mathcal{C}_t$ is exhausted).
Matched-state retrieval evaluation. Because (E_t, G_t) depend on the documents returned in earlier iterations, live runs of different rerankers diverge after the first iteration and would then no longer reorder the same $\mathcal{C}_t$. To enforce identical inputs, the *retrieval* metrics of E2–E4 are computed in matched-state mode: the MIGRES+FSR (no reranking) run is recorded once per question, storing $(q, E_t, G_t, \mathcal{C}_t)$ for every iteration t; each system then reorders exactly this stored $\mathcal{C}_t$, and the metrics are computed on its ordering. The *downstream* metrics are computed in live mode: each system is run end to end inside MIGRES with only the reranking method replaced. Retrieval metrics are also logged in the live runs as a secondary analysis. Each retrieval call (question i, iteration t) yields one ranked list and one score per metric; the score of question i is the mean over its iterations, and the question is the unit of analysis in all statistics (Section 8.10).
Retrieval metrics over iterations. The primary retrieval metrics are *per-iteration*: at each iteration t they are computed on the system's ordering of $\mathcal{C}_t$ against the full set of ground-truth relevant evidence, and evidence found at earlier iterations is not removed from that set. The per-iteration scores are then averaged over the iterations of the question. In matched-state mode the number of iterations is the one recorded in the no-reranking run, for every system. As a secondary *cumulative* measure, we also report Recall over the union of the top-k lists returned across all iterations, in which a document returned in several iterations is counted once. In the live runs the iterations actually performed by each system are used, so live retrieval metrics are not paired at the iteration level.
Primary ranking metrics. All twelve systems are evaluated with the same metrics,

Recall@k, MRR@10, nDCG@10, Precision@k

with the same k for all systems. Recall@k and Precision@k use the final depth k; MRR@10 and nDCG@10 use cutoff 10 on the system's full ordering of $\mathcal{C}_t$ and are therefore defined for every k.
Complementary metrics. We additionally report GapEvidenceCoverage@k as a GAR-specific complementary metric where the dataset annotations permit the mapping from annotations to evidence requirements; downstream answer EM; downstream answer F1; classification accuracy for datasets where accuracy is the official task metric; and supporting-fact/evidence quality where the official annotations permit it (for example the sentence-level supporting facts of HotpotQA).
Purpose of each metric.

- *Recall@k*: whether the required evidence enters the returned set;
- *MRR@10*: whether useful evidence appears very early in the ranking;
- *nDCG@10*: ranking quality when relevance has a graded structure (with binary annotations it is a position-discounted binary measure);
- *Precision@k*: how concentrated relevant evidence is in the returned set;

- *GapEvidenceCoverage@k*: whether the currently missing requirements are actually covered;
- *EM / F1 / accuracy*: whether improved retrieval translates into improved final answers.

The shared metrics are not replaced by different metrics for individual rerankers merely because their original publications used different benchmark setups (Section 8.9.2).

8.4 Experiment E3: GAR Ablation Study

The full GAR score is Eq. (7),

$$\boxed{S = \alpha R + \beta C + \gamma N.}$$

Four variants are evaluated against it.
Without relevance:

$$\boxed{S_{-R} = \beta C + \gamma N}$$

Without coverage:

$$\boxed{S_{-C} = \alpha R + \gamma N}$$

Without novelty:

$$\boxed{S_{-N} = \alpha R + \beta C}$$

Relevance-only diagnostic:

$$\boxed{GAR_{R\text{-only}} = R.}$$

Fixed conditions. All evaluation conditions are held fixed: the same FSR, candidate sets $\mathcal{C}_t$, questions, corpus, MIGRES configuration, and final k. Every variant uses the same frozen $\alpha, \beta, \gamma, K_0, k$ from the full GAR configuration (Section 6.13.5). The remaining weights are *not* renormalized after a component is removed: dividing by the sum of the retained weights would multiply every candidate's score by the same positive constant and therefore would not change the ranking. An ablation is degenerate only if the retained weights sum to zero (for example $\beta + \gamma = 0$ for S_{-R}), in which case every candidate receives the same score of zero. $GAR_{R\text{-only}}$ ranks by $R(d, q)$ alone, which is equivalent to $\alpha = 1$, $\beta = \gamma = 0$ (any positive α gives the same ranking).
Metrics. The ablations are evaluated with the same retrieval metrics (matched-state mode) and downstream metrics (live mode) as E2.
Interpretation.

- S_{-R} measures the contribution of question relevance;
- S_{-C} measures the contribution of gap coverage;
- S_{-N} measures the contribution of evidence novelty;
- $GAR_{R\text{-only}}$ provides a query-relevance baseline inside the GAR formulation. Together with S_{-N} (relevance plus coverage), it helps separate the benefit of adding gap coverage from the benefit of the full relevance–coverage–novelty combination.

8.5 Experiment E4: Candidate-Pool Sensitivity

E4 evaluates the effect of the first-stage candidate depth,

$$K_0 \in \{50, 100, 200, 500\},$$

on D_{val} only. For each K_0, the candidate set $\mathcal{C}_t$ is rebuilt by Eq. (9) and GAR is run with α, β, γ held at their validation-selected values, which isolates the effect of candidate depth. We report Recall@k, MRR@10, nDCG@10, GapEvidenceCoverage@k, reranking latency, and the number of candidates scored $|\mathcal{C}_t| \leq (m + 1)K_0$.
The value K_0 used in the final test experiments is the one selected by the Bayesian-optimization procedure of Section 6.13.4; E4 does not re-select it. E4 is a *sensitivity analysis*, not a separate final comparison: no D_{test} data are used, and no hypothesis of E2 or E3 is tested with it.

8.6 Experiment E5: Efficiency and Cost Analysis

E5 compares GAR with the reranking baselines on D_{test}, under the frozen configuration and recorded hardware, on:

- first-stage retrieval latency;
- reranking latency;
- end-to-end retrieval latency (first stage plus reranking);
- number of documents scored;
- number of model forward passes;
- number of LLM calls;
- input and output tokens;
- API cost where applicable;
- index-construction time (of FSR, and separately any reranker-specific preprocessing, training, or index);
- memory usage where applicable.

Separation of costs. Three quantities are always reported separately and never summed silently:

1. *local retrieval/reranking computation* (the FSR and the reranker, on local hardware);
2. *LLM cost of the reranker* (nonzero only for LLM-based rerankers, e.g. RankGPT);
3. *LLM cost of the surrounding MIGRES system* (gap identification, extraction, and solving).

The MIGRES cost is reported for every system, because a reranker can change the number of iterations and therefore the number of MIGRES LLM calls. Token counts, calls, and prices follow the cost model

$$\boxed{Cost(q) = \sum_{\ell \in \mathcal{L}(q)} \Big(T_{\text{in}}(\ell) p_{\text{in}}(\ell) + T_{\text{out}}(\ell) p_{\text{out}}(\ell) \Big), \qquad \overline{Cost} = \frac{1}{N} \sum_{i=1}^{N} Cost(q_i).}$$

For token accounting we additionally record

$$\boxed{T_{\text{in}}^{\text{tot}}(q) = \sum_{\ell \in \mathcal{L}(q)} T_{\text{in}}(\ell), \qquad T_{\text{out}}^{\text{tot}}(q) = \sum_{\ell \in \mathcal{L}(q)} T_{\text{out}}(\ell), \qquad T_{\text{tot}}(q) = T_{\text{in}}^{\text{tot}}(q) + T_{\text{out}}^{\text{tot}}(q).}$$

We report, for every system, the total input tokens, total output tokens, total tokens, number of API calls, and API cost over D_{test}, together with their per-question means. The raw token counts are taken from the API usage metadata when an external API is used; no token count is inferred from character length. Local model inference is counted separately as local computation and has zero API cost unless an external service is actually charged for it.

Here $\mathcal{L}(q)$ is the set of LLM calls made while answering question q, $T_{\text{in}}(\ell), T_{\text{out}}(\ell)$ are the input/output tokens of call ℓ, and $p_{\text{in}}(\ell), p_{\text{out}}(\ell)$ are the per-token prices of whichever model served that call. Because calls need not share a model or a price, the cost is summed per call; it reduces to $T_{\text{in}} p_{\text{in}} + T_{\text{out}} p_{\text{out}}$ only when $|\mathcal{L}(q)| = 1$.

Counting rules. Pointwise rerankers (MonoBERT, MonoT5, RankT5) require one forward pass per candidate; DuoBERT requires $|\mathcal{C}_t|(|\mathcal{C}_t| - 1)$ model calls (Section 3). GAR embeds the question, the evidence items in E_t, and only the documents in $\mathcal{C}_t$ with the fixed Sentence Transformer; these are local encoder passes, and gap coverage and novelty are computed locally by token overlap and

cosine similarity. **GAR itself does not require one LLM call per candidate**; in the proposed implementation its scorer makes no LLM call. GAR also has no training stage and no corpus-wide embedding index (Sections 5.2 and 6.13.1). All counts are measured by instrumentation of the running systems rather than assumed.
E5 is descriptive: costs are reported as per-question means with 95% bootstrap confidence intervals, and no significance test is part of the confirmatory protocol.

8.7 Research Questions

RQ1. How do BM25, DPR, Contriever, ColBERTv2, and SPLADE compare as first-stage retrievers on the target datasets? (E1)
RQ2. Does GAR improve retrieval quality over the same FSR candidate set without reranking? (E2)
RQ3. How does GAR compare with established reranking methods when all methods rerank the identical candidate set? (E2)
RQ4. Does improved retrieval under GAR translate into improved downstream MIGRES answer quality? (E2)
RQ5. Which GAR components—relevance, gap coverage, or novelty—contribute to its performance? (E3)
RQ6. How does GAR performance change as the first-stage candidate-pool depth K_0 changes? (E4)
RQ7. What computational and API-cost trade-offs does GAR introduce relative to the baseline rerankers? (E5)
RQ8. Does the behavior of GAR remain consistent across the six datasets? (E2, E3)

8.8 Hypotheses

Each hypothesis corresponds to one experiment. None predicts a numerical improvement, and none asserts that GAR outperforms every baseline; where the outcome is uncertain, the hypothesis is an empirical question resolved by the statistics of Section 8.10.
H1 — First-stage retrieval (E1). The five first-stage retrieval algorithms exhibit different retrieval-quality performance on the validation datasets.
H2 — GAR vs. no reranking (E2). GAR improves the primary retrieval metric (Recall@k) over the fixed FSR without reranking. This is the single confirmatory hypothesis.
H3 — GAR vs. reranking baselines (E2). GAR achieves different ranking performance from established reranking methods when evaluated on the same candidate set. The comparator for the directional comparison, the best validation-selected baseline $B^\star$ (Section 8.10), and the direction of the contrast ($\Delta = \text{GAR} - B^\star$) are fixed before D_{test} is evaluated.
H4 — Downstream QA (E2). Improvements in retrieval quality under GAR are associated with improvements in downstream MIGRES answer quality.
H5 — Coverage contribution (E3). Removing the coverage component decreases gap-evidence coverage and/or overall retrieval quality.
H6 — Novelty contribution (E3). Removing the novelty component increases redundancy and decreases retrieval quality or evidence coverage.
H7 — Relevance contribution (E3). Removing the relevance component decreases ranking quality with respect to the original question.
H8 — Candidate-pool sensitivity (E4). GAR performance changes as K_0 changes, with larger candidate pools potentially increasing computational cost.
Operationalization. H1: pairwise differences among the five retrievers on D_{val}, on CandRecall@K_0^{sel}. H2: GAR vs. FSR on Recall@k. H3: GAR vs. each reranker on Recall@k. H4: GAR vs. FSR on the official answer metric, and the association between per-question retrieval change and per-question answer-metric change (Section 8.10). H5: Full GAR vs. S_{-C} on GapEvidenceCoverage@k and on Recall@k; supported if a decrease is established on at least one of them. H6: Full GAR vs. S_{-N}; supported if S_{-N} has higher Redundancy@k (Section 8.9.3) *and* lower Recall@k or lower GapEvidenceCoverage@k. H7: Full GAR vs. S_{-R} on MRR@10 and nDCG@10, together with

Recall@k. H8: GAR on D_{val} at each K_0 against the selected K_0, together with the measured reranking cost.

8.9 Primary and Secondary Metrics

8.9.1 Metric hierarchy

Evaluation level	Primary metrics	Secondary metrics
First-stage retrieval	CandRecall@K_0^{sel} (Eq. (9) pool)	Question-only Recall@k, MRR@10, nDCG@10; mean pool size; latency
Reranking	Recall@k, MRR@10, nDCG@10	Precision@k, GapEvidenceCoverage@k
Downstream QA	Official dataset answer metric	EM, F1, accuracy, supporting-fact quality where applicable
Efficiency	End-to-end latency	Memory, indexing time, documents scored, tokens, API cost

The same ranking metrics are applied to *all* rerankers, regardless of which metrics were used in their original publications. Of the three primary ranking metrics, Recall@k is the single pre-registered confirmatory metric (Section 8.10); it is also the objective used to select GAR's hyperparameters on D_{val} (Section 6.13.4). MRR@10 and nDCG@10 are reported with the same intervals but as secondary contrasts.

8.9.2 Motivation from the original publications

Before fixing the suite, we checked which metrics the primary publications of the compared methods actually report. Table 1 records what was verified; it is used only to justify the common suite, not to reproduce each paper's setup.

Method (source)	Ranking / QA metrics reported in the original publication
MonoBERT [**??**]	MRR@10 (MS MARCO passage); MAP (TREC-CAR).
MonoT5 [**?**]	MRR@10 (MS MARCO passage); MAP and nDCG@20 (zero-shot TREC Robust04).
DuoBERT [**?**]	MRR@10 (MS MARCO); MAP (TREC-CAR).
ColBERTv2 [Santhanam et al., 2022]	MRR@10 and Recall@k (MS MARCO, in domain); nDCG@10 (BEIR); Success@5 (LoTTE).
LambdaMART [Burges, 2010]	Algorithmic overview without a benchmark metric suite; the method is presented as optimizing rank measures such as NDCG.
RankT5 [Zhuang et al., 2023]	MRR@10 and nDCG@5, nDCG@10 (MS MARCO and Natural Questions dev sets, reranking the top-1000 of a dual encoder).
MMR [Carbonell and Goldstein, 1998]	Two-page poster with preliminary retrieval and summarization results; no standardized ranking-metric suite.
AIR [Yadav et al., 2020]	Evidence selection: precision, recall, and F1 (MultiRC) and Recall@10 (QASC); downstream QA performance.
Dartboard [Pickett et al., 2024]	nDCG for the retrieval component and end-to-end QA correctness (Retrieval-Augmented Generation Benchmark).
RankGPT [**?**]	nDCG@1, nDCG@5, nDCG@10 (TREC, BEIR, Mr. TyDi).

Table 1: Metrics reported by the original publications of the compared methods, as verified from the primary publications for this study.

The publications use different benchmarks, cutoffs, and metric sets, so reproducing every original metric would make the cross-method comparison incoherent. We therefore adopt a common ranking core that each family of methods already supports:

- **MRR@10** is the headline metric of the BERT- and T5-based rerankers (MonoBERT, DuoBERT, MonoT5, RankT5) and is also reported for ColBERTv2;
- **nDCG@10** is reported by RankT5, ColBERTv2 (BEIR), and RankGPT (which also reports nDCG@1 and nDCG@5), and nDCG is the retrieval metric of Dartboard; it also accommodates graded relevance where annotations provide it;
- **Recall@k** is reported by ColBERTv2 and AIR (Recall@10) and is the metric most directly aligned with GAR's purpose, retrieving evidence that is still missing;
- **Precision@k** is a complementary measure, consistent with the precision-based evidence-selection evaluation of AIR;
- **Downstream answer metrics** (EM, F1, accuracy, supporting-fact quality) follow the practice of AIR and Dartboard of assessing retrieval by its effect on question answering, and are needed because MIGRES is evaluated end to end.

MAP, nDCG@20, nDCG@1/5, and Success@k appear in individual papers but are not part of the common suite; they can be computed offline from the saved rankings as supplementary analyses without changing any confirmatory result.

8.9.3 Metric definitions

The unit of retrieval is the document unit used in the MIGRES corpus. Relevance judgments come from each dataset's official evidence annotations, as recorded in the experiment configuration (Section 7); they are binary unless the annotation is graded, and the ground truth is used only for scoring.

Recall@k. Recall@k measures how much of the ground-truth relevant evidence appears in the first k retrieved documents:

$$\boxed{Recall@k = \frac{|\text{relevant evidence} \cap \text{top-}k|}{|\text{relevant evidence}|}}$$

Benchmarks used here provide at least one relevant-evidence item per question, so this denominator is not expected to be zero in practice; if an instance with no annotated relevant evidence does occur, that instance is excluded from the Recall@k average (for all systems alike) rather than reported as 0/0.

Candidate-pool recall (E1 only). CandRecall@K_0 is the fraction of ground-truth relevant evidence contained in the full candidate set $\mathcal{C}_t^{\rho}$ built by Eq. (9) at per-query depth K_0 for a candidate retriever ρ (Section 8.2); it is computed per iteration and averaged over the iterations of a question, and the same exclusion rule for questions without annotated evidence applies. It is a property of an unordered set, so it is not a ranking metric.

Precision@k.

$$\boxed{Precision@k = \frac{|\text{relevant documents in top-}k|}{k}}$$

MRR and MRR@10. Mean Reciprocal Rank measures how early the first relevant result appears:

$$\boxed{MRR = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{rank_i}}$$

where $rank_i$ is the rank of the first relevant result for question i. For MRR@10, $\frac{1}{rank_i}$ is set to 0 whenever no relevant document appears in the first 10 results, following standard practice.

nDCG@k. For graded relevance:

$$DCG@k = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)}, \qquad \boxed{nDCG@k = \frac{DCG@k}{IDCG@k}}$$

where $IDCG@k$ is the ideal DCG at cutoff k; nDCG@10 is the case $k = 10$. The cumulated-gain idea is due to Järvelin and Kekäläinen [2002]; the exponential-gain form $2^{rel_i} - 1$ with a $\log_2(i+1)$ discount used here is the variant common in learning to rank [**?**], and differs from the linear-gain, $\log_b$-discount definition of the original paper. If $IDCG@k = 0$ (no relevant document exists for that question), we define $nDCG@k = 0$ for that question rather than leaving the ratio undefined.

GapEvidenceCoverage@k. Because GAR explicitly models an information gap, we additionally report an evidence-coverage measure where the dataset annotations permit it:

$$\boxed{GapEvidenceCoverage@k = \frac{\text{ground-truth evidence requirements retrieved in top-}k}{\text{total ground-truth evidence requirements}}}$$

The exact mapping from benchmark annotations to evidence requirements is defined separately for each dataset and recorded in the experiment configuration before D_{test} is evaluated; for a dataset where no such mapping is possible the metric is not reported. As with Recall@k, an instance with zero total ground-truth evidence requirements is excluded from the average rather than reported as 0/0.

Redundancy@k (diagnostic for H6 only). H6 concerns redundancy, which none of the metrics above measures. For a returned set T and evidence $E_t \neq \varnothing$ we define, with the token normalization $\tau(\cdot)$ of Eq. (3),

$$Redundancy@k = \frac{1}{|T|} \sum_{d \in T} \max_{e \in E_t} \frac{|\tau(d) \cap \tau(e)|}{|\tau(d) \cup \tau(e)|}, \qquad T = \text{top-}k.$$

It is a lexical overlap measure, deliberately different from the embedding-based novelty term of Eq. (5) so that it does not favor GAR by construction. It is undefined when $E_t = \varnothing$ (first iteration), and such calls are excluded. It is used only for the ablation contrast of H6 and is not part of the ranking metric suite.

8.9.4 Downstream QA metrics

The final MIGRES answer is evaluated with the metrics supported by each benchmark, using the official evaluation script whenever one is available: Exact Match (EM); token-level F1; answer accuracy for classification-style datasets; and evidence/supporting-fact quality where annotations permit. For a benchmark with a normalized answer set A_i,

$$EM_i = \mathbf{1}[\hat{a}_i \in A_i], \qquad EM = \frac{1}{N} \sum_{i=1}^{N} EM_i, \qquad F1 = \frac{2PR}{P+R},$$

where P and R are the precision and recall over normalized answer tokens. $F1$ is undefined when $P + R = 0$; we adopt the convention of the benchmark's official evaluation script for this case (typically $F1 = 1$ if both predicted and gold answers are empty after normalization and $F1 = 0$ if only one is). The *official dataset answer metric* of Section 8.9 is the answer metric that dataset's official evaluation designates.

8.10 Statistical Analysis

We define a single pre-registered statistical protocol. This section, the frozen experiment configuration, the identity of the comparator $B^\star$, and the list of tests in Table 2 are recorded before any D_{test} result is inspected.

Paired design. All systems evaluate the same questions, so every comparison is paired at the level of the question. Let $m_i(\cdot)$ be the per-question value of a metric (Section 8.3); for systems A and B the paired difference is $d_i = m_i(A) - m_i(B)$ and $\Delta = \frac{1}{N}\sum_i d_i$. A question that is excluded from a metric (for example because it has no annotated evidence) is excluded for all systems.
Paired bootstrap. Every test uses a paired bootstrap with $B = 10{,}000$ resamples: questions are resampled with replacement (the same indices for both systems), and Δ_b^* is recomputed on each resample, $b = 1, \ldots, B$, with a fixed and recorded random seed. Pooled analyses across the six datasets resample questions within each dataset and use the macro-average of the per-dataset Δ.

- *Significance threshold:* $p < \alpha_{\text{sig}} = 0.05$.
- *95% confidence interval:* the 2.5th and 97.5th percentiles of $\{\Delta_b^*\}$.
- *Effect size:* the mean paired difference Δ in metric units, and the standardized paired effect size $d_z = \bar{d}/s_d$ (mean over standard deviation of the d_i), each with a 95% bootstrap interval.
- *p-values:* for a one-sided test of $A > B$, $p = (1 + \#\{b : \Delta_b^* \le 0\})/(B+1)$; for a two-sided test, $p = \min\{1, 2\min(p_{\le}, p_{\ge})\}$ with $p_{\le}, p_{\ge}$ the one-sided values.

Primary (confirmatory) comparison.

$$\text{GAR} \quad \textit{vs.} \quad \text{FSR (no reranking)}$$

on the primary retrieval metric, Recall@k, pooled across the six datasets, tested one-sided ($H_1 : \Delta > 0$) at $\alpha_{\text{sig}} = 0.05$ with no adjustment. It is the only confirmatory test.
Secondary comparison against the best baseline. $B^\star$ is the reranker with the highest macro-averaged Recall@k on D_{val} among the ten rerankers of Section 3, with each reranker's free hyperparameters set to its official defaults or tuned on D_{val} only, as recorded in the configuration. $B^\star$ is frozen before D_{test} is evaluated. The comparison GAR vs. $B^\star$ uses the same paired-bootstrap framework, two-sided, with $\Delta = \text{GAR} - B^\star$. **The test set is never used for model, hyperparameter, or comparator selection.**
Multiple comparisons. All secondary contrasts and ablations form one family (Table 2); the Holm step-down procedure controls the family-wise error rate at $\alpha_{\text{sig}} = 0.05$. With the m two-sided p-values ordered $p_{(1)} \le \cdots \le p_{(m)}$, the adjusted values are $\tilde{p}_{(j)} = \max_{i \le j} \min\{1, (m - i + 1)\, p_{(i)}\}$, and a contrast is declared significant if $\tilde{p} < 0.05$. Both raw and adjusted p-values are reported. Holm is applied separately to the pooled analysis and within each dataset; m is the number of tests actually performed (a test whose metric is unavailable for a dataset is omitted from that dataset's family).

ID	Contrast	Metric(s)	Hyp.	Number / status
T0	GAR vs. FSR	Recall@k	H2	1, confirmatory
S1	GAR vs. $B^\star$	Recall@k	H3	1
S2	GAR vs. each of the other ten rerankers	Recall@k	H3	10
S3	GAR vs. FSR	MRR@10, nDCG@10, official answer metric	H2, H4	3
S4	Full GAR vs. S_{-C}	Recall@k, GapEvidenceCoverage@k	H5	2
S5	Full GAR vs. S_{-N}	Recall@k, GapEvidenceCoverage@k, Redundancy@k	H6	3
S6	Full GAR vs. S_{-R}	Recall@k, MRR@10, nDCG@10	H7	3
S7	Full GAR vs. $GAR_{R\text{-only}}$	Recall@k	(E3)	1

Table 2: Pre-registered test list. T0 is confirmatory; S1–S7 form the Holm family (24 tests per analysis unit when every metric is available).

Validation-set analyses. E1 and E4 are run on D_{val} and are used for selection and sensitivity, not for confirmatory claims. For H1, the ten pairwise contrasts among the five retrievers are tested two-sided on CandRecall@K_0^{sel} with Holm correction. For H8, GAR at each $K_0 \in \{50, 100, 200, 500\}$ is compared with GAR at the selected K_0 on Recall@k (Holm over the three contrasts), and curves with 95% intervals are reported for all E4 metrics.
Association for H4. In addition to S3, we report the Spearman rank correlation between the per-question change in Recall@k (GAR minus FSR) and the per-question change in the official answer metric, with a 95% bootstrap interval. It is descriptive and reported as an association, not a causal claim.
Consistency across datasets (RQ8). Per-dataset Δ, its 95% interval, and the sign of the effect are reported for every dataset and every contrast, alongside the pooled result; consistency is judged from the agreement in sign and interval across the six datasets, not from a further test.

8.11 Experimental Fairness and Reproducibility

The following remain fixed across all systems:

- dataset;
- split;
- question set;
- corpus;
- FSR;
- K_0, except in the dedicated candidate-pool sensitivity study (E4);
- final k;
- MIGRES prompts;
- gap-generation procedure;
- extraction procedure;
- answer-generation model;
- stopping conditions;
- evaluation scripts;
- random seeds where applicable.

For the main reranking comparison, only the reranking method changes. Ground-truth answers and evidence are never used to construct G_t, E_t, or $\mathcal{C}_t$. Any baseline-specific training or hyperparameter setting follows the protocol of Section 3.14, is documented, uses no test annotations, and is fixed before D_{test} is run. Where a component is stochastic (for example an LLM-based reranker), its sampling settings and seeds are fixed in advance and recorded. The candidate-pool depth K_0 is validation-selected for GAR and shared with all baselines, which is a limitation of the comparison discussed in Section 6.13.5.
We record exact model checkpoints, library versions, hardware, retrieval parameters, and reranker configurations, as listed below.

Parameter	Value
Dataset	Exact release and split
Corpus	Exact corpus version
First-stage retriever (FSR)	Selected empirically by E1 (Section 8.2); fixed for all systems thereafter
Reranker	None / MonoBERT / MonoT5 / DuoBERT / ColBERTv2 / LambdaMART / RankT5 / MMR / AIR / Dartboard-RIG / RankGPT / GAR
Baseline checkpoint and configuration	Exact model/checkpoint and hyperparameters, per Section 3.14
Adapted baselines	AIR, Dartboard/RIG, RankGPT adaptations as recorded
FSR selection depth K_0^{sel}	100 (pre-specified, not tuned)
E1 reference trajectory	Recorded MIGRES+BM25 run of (q, E_t, G_t) on D_{val}
D_{val} / D_{test} construction	Official splits, or seeded halves of the dev split where test labels are withheld
Text preprocessing for $\tau(\cdot)$	NFKC, lowercase, `\w+` tokens, NLTK stopwords, Porter stemming
Embedding input length	512 word pieces, tail truncation; truncation rate reported
GAR embedding model	`multi-qa-mpnet-base-cos-v1`
Final retrieval depth k	User-configured positive integer; fixed across all systems in a comparison
FSR retrieval depth K_0 (per query)	Validation-selected GAR value
α, β, γ	Validation-selected values
Bayesian optimization seed	Recorded value
Bootstrap seed and resamples	Recorded seed; $B = 10{,}000$
FSR implementation	Exact library/index
Hardware	CPU/GPU configuration
Software	Python and package versions
API pricing configuration	Model-specific input/output prices, currency, and price-date snapshot used for E5
MIGRES configuration	Exact prompts/model/settings
Pre-registration record	Test list, $B^\star$, and metric-to-annotation mappings, fixed before D_{test}

8.12 Final Experimental Matrix

Experiment	Systems	Data	Main purpose
E1	BM25, DPR, Contriever, ColBERTv2, SPLADE	D_{val}	Select FSR
E2	FSR, MonoBERT, MonoT5, MonoT5-G, DuoBERT, ColBERTv2, LambdaMART, RankT5, MMR, AIR, Dartboard/RIG, RankGPT, GAR	D_{test}	Main reranking comparison
E3	GAR and component ablations (S_{-R}, S_{-C}, S_{-N}, $GAR_{R\text{-only}}$)	D_{test}	Component contribution
E4	GAR with $K_0 \in \{50, 100, 200, 500\}$	D_{val}	Candidate-pool sensitivity
E5	GAR + all relevant baselines	D_{test}	Efficiency and cost

9 Conclusion

This project proposes Gap-Aware Retrieval (GAR) as a reranking algorithm for iterative information-seeking systems, applied on top of a first-stage retriever's candidates. GAR is evaluated inside MIGRES rather than replacing the MIGRES framework itself.
The core idea is:

$$\boxed{\text{Relevant} + \text{Gap-covering} + \text{Non-redundant}}$$

implemented as:

$$\boxed{S(d \mid q, E_t, G_t) = \alpha R(d, q) + \beta C(d, G_t) + \gamma N(d, E_t)}$$

The complete pipeline is:

First-Stage Retrieval → Candidate Set → Relevance/Novelty Embeddings + Gap-Coverage Token Overlap → GAR Scoring → Top-k → MIGRES

GAR has no training stage in the primary experiment. Its scoring hyperparameters are selected on D_{val} through the complete MIGRES+FSR+GAR pipeline using Bayesian optimization and then frozen for D_{test}.
The central empirical question is whether reranking with the information gap and previously collected evidence produces a more useful ranking than standard reranking algorithms applied to the same FSR candidates.

References

Keheng Wang, Feiyu Duan, Peiguang Li, Sirui Wang, and Xunliang Cai. LLMs know what they need: Leveraging a missing information guided framework to empower retrieval-augmented generation. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 2379–2400. Association for Computational Linguistics, 2025. URL https://aclanthology.org/2025.coling-main.163/.

Vikas Yadav, Steven Bethard, and Mihai Surdeanu. Unsupervised alignment-based iterative evidence retrieval for multi-hop question answering. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4514–4525. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.414. URL https://aclanthology.org/2020.acl-main.414/.

Marc Pickett, Jeremy Hartman, Ayan Kumar Bhowmick, Raquib-ul Alam, and Aditya Vempaty. Better rag using relevant information gain. *arXiv preprint arXiv:2407.12101*, 2024. URL https://arxiv.org/abs/2407.12101.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1410. URL https://aclanthology.org/D19-1410/.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *arXiv preprint arXiv:1702.08734*, 2017. URL https://arxiv.org/abs/1702.08734.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019. URL https://doi.org/10.1561/1500000019.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-main.550. URL https://aclanthology.org/2020.emnlp-main.550/.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 2665–2679. Association for Computational Linguistics, 2022. URL https://aclanthology.org/2022.emnlp-main.738/.

Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. ColBERTv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3715–3734. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.naacl-main.272. URL https://aclanthology.org/2022.naacl-main.272/.

Thibault Formal, Carlos Lassance, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for first stage ranking. *arXiv preprint arXiv:2107.05720*, 2021. URL https://arxiv.org/abs/2107.05720.

Christopher J. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010.

Jaime Carbonell and Jade Goldstein. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 335–336, 1998. doi: 10.1145/290941.291025. URL https://doi.org/10.1145/290941.291025.

Honglei Zhuang, Zhen Qin, Rolf Jagerman, Kai Hui, Ji Ma, Jing Lu, Jianmo Ni, Xuanhui Wang, and Michael Bendersky. RankT5: Fine-tuning T5 for text ranking with ranking losses. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2308–2313, 2023. doi: 10.1145/3539618.3592047. URL https://arxiv.org/abs/2210.10634.

Sentence Transformers. multi-qa-mpnet-base-cos-v1. Hugging Face model card, 2020. URL https://huggingface.co/sentence-transformers/multi-qa-mpnet-base-cos-v1. 768-dimensional sentence-transformers model designed for semantic search.

Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25, 2012. URL https://arxiv.org/abs/1206.2944.

Johannes Welbl, Pontus Stenetorp, and Sebastian Riedel. Constructing datasets for multi-hop reading comprehension across documents. *Transactions of the Association for Computational Linguistics*, 6:287–302, 2018. doi: 10.1162/tacl_a_00021. URL https://aclanthology.org/Q18-1021/.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380. Association for Computational Linguistics, 2018. doi: 10.18653/v1/D18-1259. URL https://aclanthology.org/D18-1259/.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022. doi: 10.1162/tacl_a_00475. URL https://aclanthology.org/2022.tacl-1.31/.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL https://aclanthology.org/Q19-1026/.

Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pages 1601–1611. Association for Computational Linguistics, 2017. doi: 10.18653/v1/P17-1147. URL https://aclanthology.org/P17-1147/.

Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies. *Transactions of the Association for Computational Linguistics*, 9:346–361, 2021. doi: 10.1162/tacl_a_00370. URL https://aclanthology.org/2021.tacl-1.21/.

Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of ir techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002. doi: 10.1145/582415.582418. URL https://doi.org/10.1145/582415.582418.